Modular Multiplayer FPS Engine

Version 0.5.6.0

Table of contents

- Design Principles of the kit
  - Abstract classes
  - Scriptable objects
  - Input System
- Project Setup
  - First Time Setup
  - Layers
  - Tags
- Mobile Controls
  - Tips
    - Performance
- Using the kit
  - Setting up a new scene
  - Adding a new weapon
  - Using the attachments system
  - Setting up spawns
  - Setting up a new player model
  - Modifying the UI
  - Update Photon
  - Adding a new third person animation set
  - Adding new Footsteps
  - Terrain Footsteps
- Extended Information
  - Weapon Setup
  - Minimap
    - Setting up a scene for the minimap
  - Bots
    - How they work
    - Default AI Behaviour
  - Game Modes
    - Domination
- Platforms
  - WebGL

- Notes
- Guide
  - Color Space
  - Photon Voice
  - Compiling
  - Running your build
- WebGL FAQ
  - My build crashes with an invalid function pointer error
  - What is the memory requirement?
- Special Controls
  - Leaning
  - Voice Chat

Integrations
- Photon Voice
- First Person View 2
  - Getting started
  - Enabling the integration
  - Changing shaders
  - Modifying the prefabs
  - Done
- Steam
  - Getting Started
    - Main
    - Friend Menu
  - Using your own AppID
  - Features

Recommended Assets
- Animations

FAQ Help
- My weapons are clipping through level geometry?!
- The Loadout Dropdown Buttons are rotated.
- How do I scale, rotate or move a weapon prefab (first, third-person or drop prefab)?
- I replaced the hit indicator. Now it is not centered anymore.
- [...] is inaccessible due to its protection level.
- My snipers are inaccurate.
- I spawn as a construction worker.
- My customization UI Dropdowns are super small / super big

Need help?

Design Principles of the kit

Abstract classes

As the name suggests, the kit is built on a modular design. To make this possible, there are a lot of base classes that are abstract, which means that they cannot function on their own and need to be overridden by a non abstract class, which can then function.

Scriptable objects

To make the kit cleaner, a lot of the so called modules are based on ScriptableObject instead of MonoBehaviour, which means that they do not exist in the game hierarchy but instead in the project. This means that runtime data, while it can, should not be saved inside them, because it is shared with all scripts that access that specific scriptable object. To solve this issue, a normal class, not a MonoBehaviour, can be saved in specific object variables in a MonoBehaviour (usually this would be Kit_PlayerBehaviour or Kit_IngameMain). Another great thing about this way is that fixed variables (such as e.g. BulletsPerMag) will not take up more memory than needed! If it was saved in the MonoBehaviour, it would take up memory each time it is instantiated. You can learn more about ScriptableObjects here and here.

Input System

Introduced for bots, the input for the whole player is stored in a class (“Kit_PlayerInput”). This allows the bots to use the same scripts as the players do. The downside is that these are only of type ‘hold’ and things such as GetKeyDown need to be coded manually using an additional variable and comparing if the state changed.

Project Setup

First Time Setup

The project comes for Unity 2017.1.5f1 but will also work in other (later) versions.
Before you can start using & modifying the kit, you will need to set the project up for the first time. To do so, you need to have a Photon AppId, which you can get here by signing

up. When you have one, click on “Window” in the Unity toolbar, then on “Photon Unity Networking” and finally on “PUN Wizard”. After the wizard is open, click on “Setup Project” and follow the wizard. When it is done, you will also need to add your AppID to the “Voice AppId” in the “PhotonServerSettings” file (“MarsFPSKit\Required\Photon\PhotonUnityNetworking\Resources”). When that is done, you are ready to use the kit! Open the Main Menu in “314 Arts\MarsFPSKit\Scenes” and start to play around!

Layers

- User Layer 8: PlayerRoot
  - This layer contains only the root of the Player object. It is used so that raycasts would not hit the root collider (CharacterController) of the player.
- User Layer 9: PlayerColliders
  - This layer contains the active colliders of the player models (not of the ragdolls) so that they can ignore collisions with PlayerRoot.
- User Layer 10: IgnoreCollisionWithPlayer
  - Everything in this layer will not collide with the player.
- User Layer 11: PlayerRagdoll
  - This contains all objects of the Ragdolls so that they will not collide with the player, as they are not synced (Start position is, but Unity does not have deterministic physics, so all clients will produce different results)
- User Layer 12: LoadoutMenu
  - The default loadout menu uses this layer so that the objects may never be rendered by the scene camera

Tags

- Tag 0: PlayerCollider
  - Active player colliders (the same which are in User Layer 9) should have this tag.
- Tag 1: Concrete
  - A misc. Tag that will cause the concrete hit particles to be played (they will play by default)
- Tag 2: Dirt
  - A misc. Tag that will cause the dirt hit particles to be played.
- Tag 3: Metal
  - A misc. Tag that will cause metal the hit particles to be played.
- Tag 4: Wood
  - A misc. Tag that will cause the wood hit particles to be played.
- Tag 5: Blood
  - A misc. Tag that will cause the blood hit particles to be played.

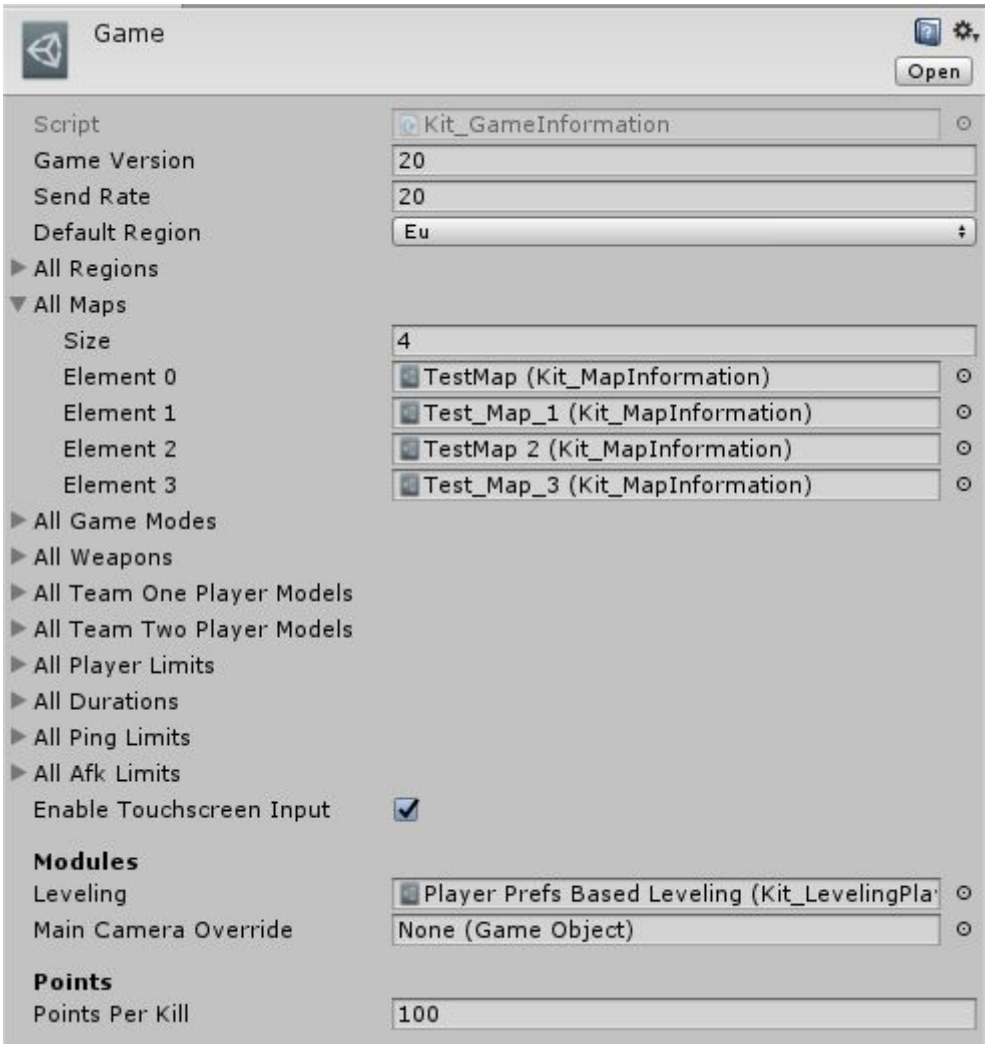
Mobile Controls

MMMPFPSK features built in Mobile controls. In a few steps, you can have it converted to mobile.

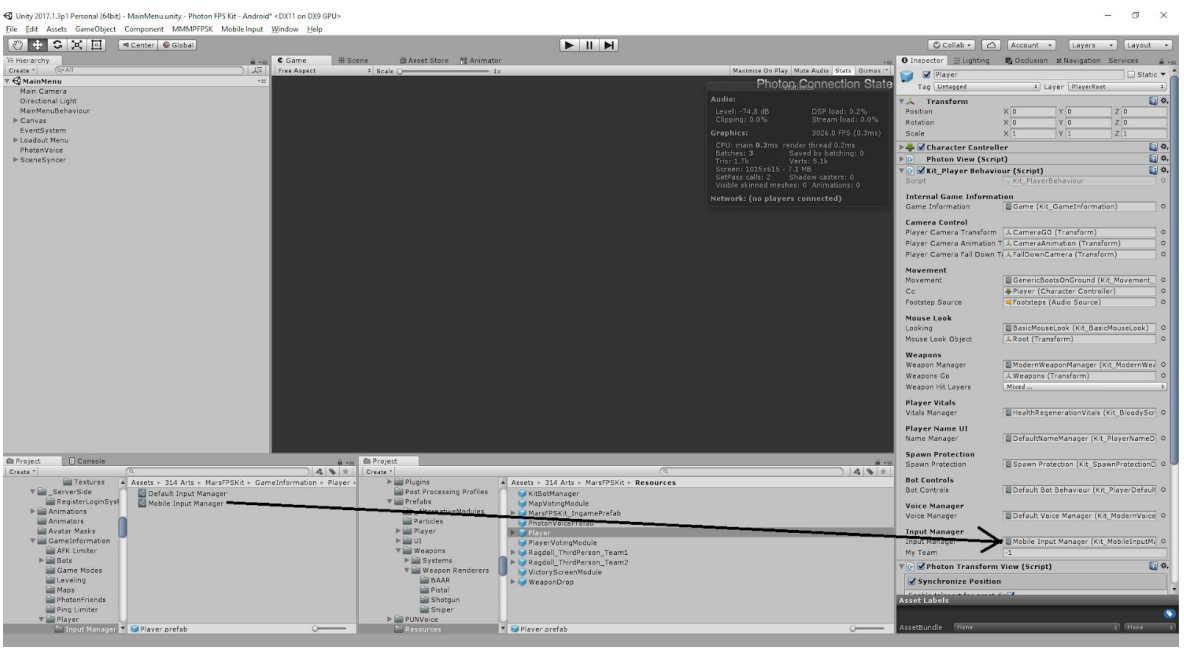
NOTE: Maps, Weapons, Character models are <u>not</u> optimized for mobile use.

First, change the Platform to Android / iOS (depending on what you want to build for).

Then, we need to enable touchscreen input, in the “Game” file, which can be found in “314 Arts\MarsFPSKit\GameData”, just check “Enable Touchscreen Input”:



Next, we need to set up the mobile controls.To do that, we need to assign the “Mobile Input Manager”, which can be found in “314 Arts\MarsFPSKit\GameData\Player\Input Manager” to the Player prefab, which can be found in “\314 Arts\MarsFPSKit\Resources”, like that:



The Kit will take care of the rest, once that is done.

Tips

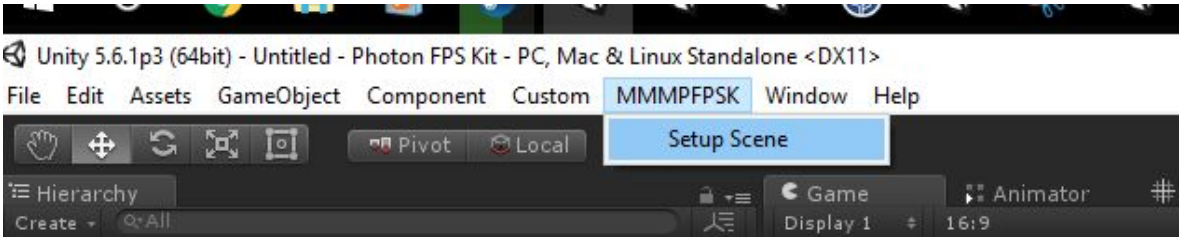
Performance

For performance reasons, you should lightmap (bake, not realtime) all your scenes. In addition, you should turn off any image effects and try to have your weapons in one part and keep batches in your level < 100.
Another important thing is, your renderer should be set to “Forward”. You can do that by modifying the camera in the MarsFPSKit_IngamePrefab prefab.

Using the kit

Setting up a new scene

Setting up a new scene for use is very easy. Just click on “MMMPFPSK/Setup scene” and save the scene if asked.



This will save the scene, add the MarsFPSKit_IngamePrefab prefab and add it to the build settings. Afterwards, you will need to add the map to the Game Modes (default: DM, TDM, GG, DOM), to lobby and/or traditional (you do not need to add it to all game modes, that's what the system is in place for. Then you can edit the MapInformation and ingame settings as you like. You can also do these steps manually if you would rather do that.

Adding a new weapon

Prequisites:

- An animated weapon (Draw, Putaway, Reload and Fire are the animations needed at the very least; More animations like Fire Last, Fire Aim, Reload Empty, Shotgun reload can be used)
- Fitting sounds
- The weapon model separately (For third person / drop). If necessary you could also use the one out of the animation files. I recommend to use a model with less detail than the first person one.

To add a new weapon, you first need to set up the prefabs. The variables are mostly self explaining. Put all non attachment renderers into the "All Weapon Renderers" array.
In the First Person Prefab, all renderers should be set to not cast shadows.
Regarding animations:

- Need to be on a generic rig (Mecanim)
- Duplicate the "FP_Generic" animator by clicking on it and pressing (Ctrl + D)
- Assign your new animations inside the animator. It is important that you do not change the names of the Mecanim States
- If you don't have all needed animations (Like Fire Last / Fire Aim) you can also use other animations such as normal Fire or leave it out (If you don't have a Dry Fire animation, you should leave that blank)
- Assign the animator to the instance of your model, and assign that Animator to the Kit_WeaponRenderer
- For reference, see the default weapons
- I recommend duplicating one of the default weapons so you can use the muzzle flash of that one and also keep the Run Pos / Run Rot settings

Do the same with the Third Person Prefab and the Drop prefab. If you already want to set up attachments, make sure that all three prefabs line up (The weapon information will tell you if that is not the case).
Once the prefabs are done, go into the Weapons folder (By default: "MarsFPSKit/GameData/Player/Weapons") and select the appropriate category (You can add / remove / modify however you like, it is just for organizing it). Either create a new Kit_ModernWeaponScript or duplicate one. I strongly recommend to duplicate one because it is easier. Assign the Prefabs and assign your sounds on the **Behaviour**. Once you are done with that, you can add the Behaviour to the GameData (By default: "MarsFPSKit/Resources/Game.asset"). The array's name is: "All Weapons". Once you added that, your weapon is in game. Now you can start tweaking aim position, left hand IK position and so on to your liking. Note that some things (such as RPM) will not update while you are playing, to save some time on calculating.

Using the attachments system

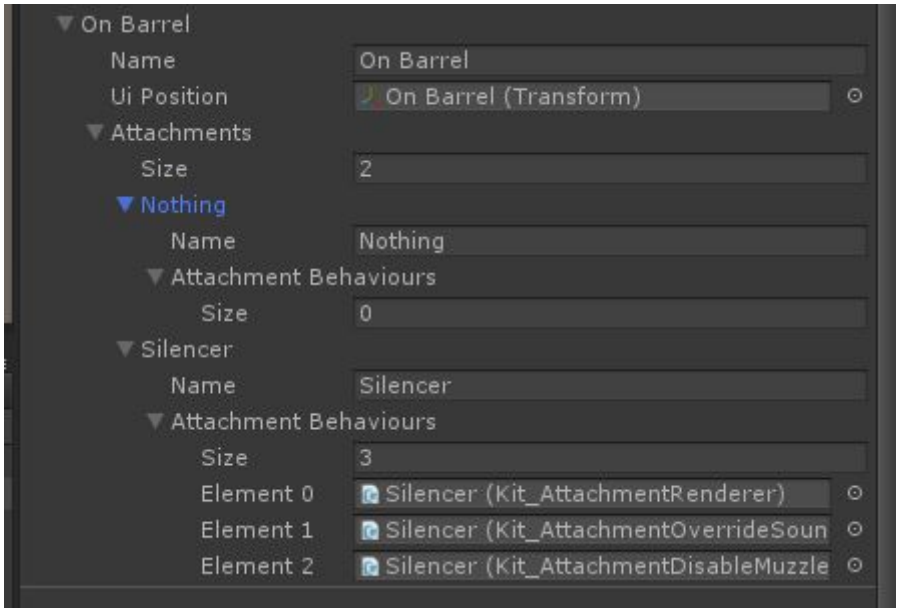
The attachments of a weapon are controlled purely by the assigned prefabs (First Person, Third Person and Drop).
Attachments are split up in two parts:

1. Attachment Slots
2. Attachments

A slot is something like sights, on barrel, etc..
An attachment is a child of a slot, it would be a Red Dot or a suppressor.
To add a new slot, simply extend the array “Attachment Slots” on all three prefabs and adjust the new slot name and UI position to your liking.



To add a new attachment, simply extend the “Attachment” array on the appropriate slot and add the behaviours you want for it.



Extended information on this subject can be found here.

Setting up spawns

In order to add a new spawn, you need to place a game object in the game world where you want the spawn to be. When that is done, add the “Kit_PlayerSpawn” script to it.
In the “Game Modes” array, add all the Game Mode Behaviours, which you want the spawn to be enabled for. There should be at least one game mode assigned, otherwise the spawn will be useless. If you want to use the spawn for all game modes, you will need to assign all game modes to them (that is how the spawns in the example scenes are setup).

If the game mode supports multiple spawn groups (for example that could be in a domination type game mode, where 0 could be normal spawns, and 1 - 3 would be spawns for each flag). If your game mode does not use Spawn Group IDs, just leave it set to 0.
Don't forget to save the scene and then you are done with setting up a new spawn. The game mode behaviours will automatically find the right spawns, so **LEAVE THE GAME OBJECT ACTIVE!**
A red gizmo box will be displayed where the spawn is. If you want to hide them, click on the script and collapse it in the inspector.

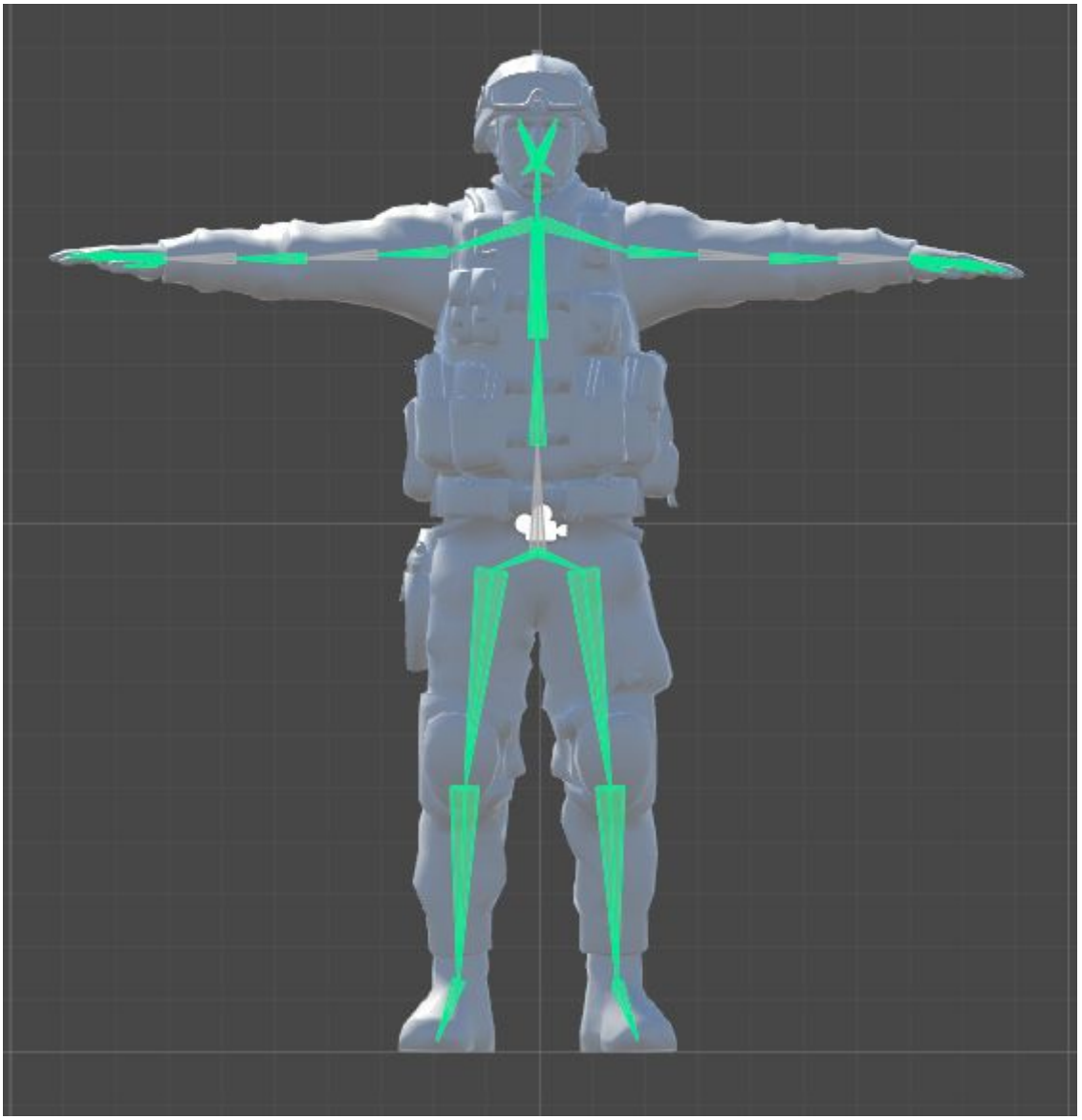
Setting up a new player model

Prequisites:

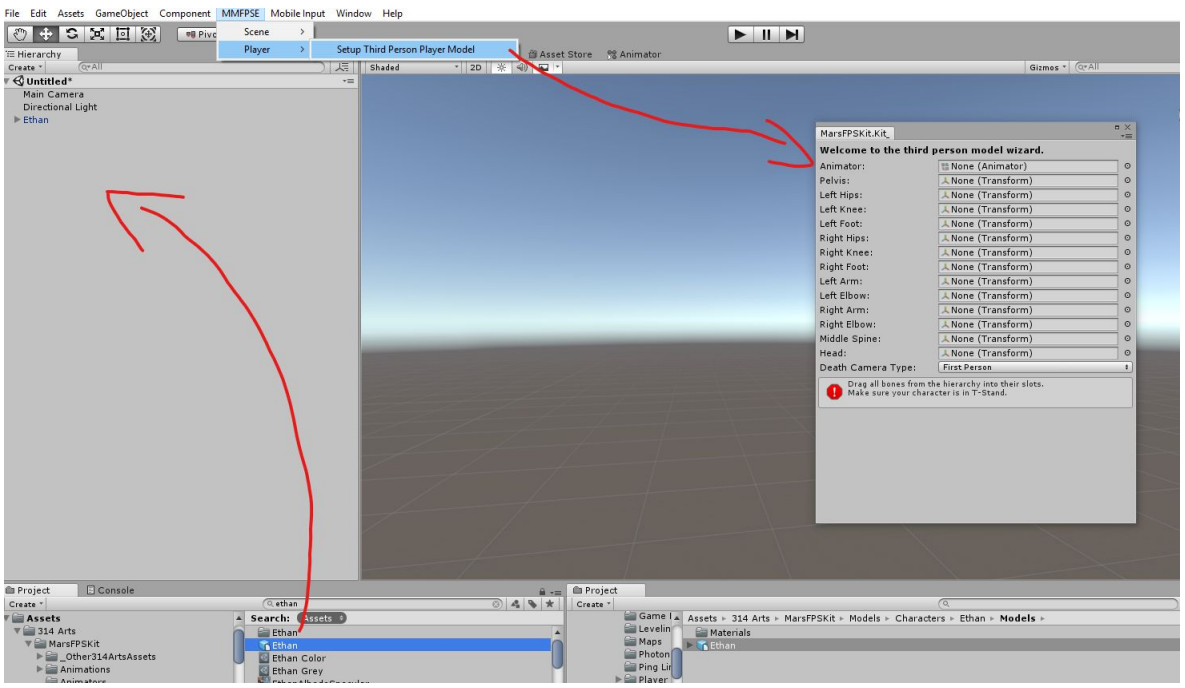
- A rigged, bipedal 3d model

Preparing the model

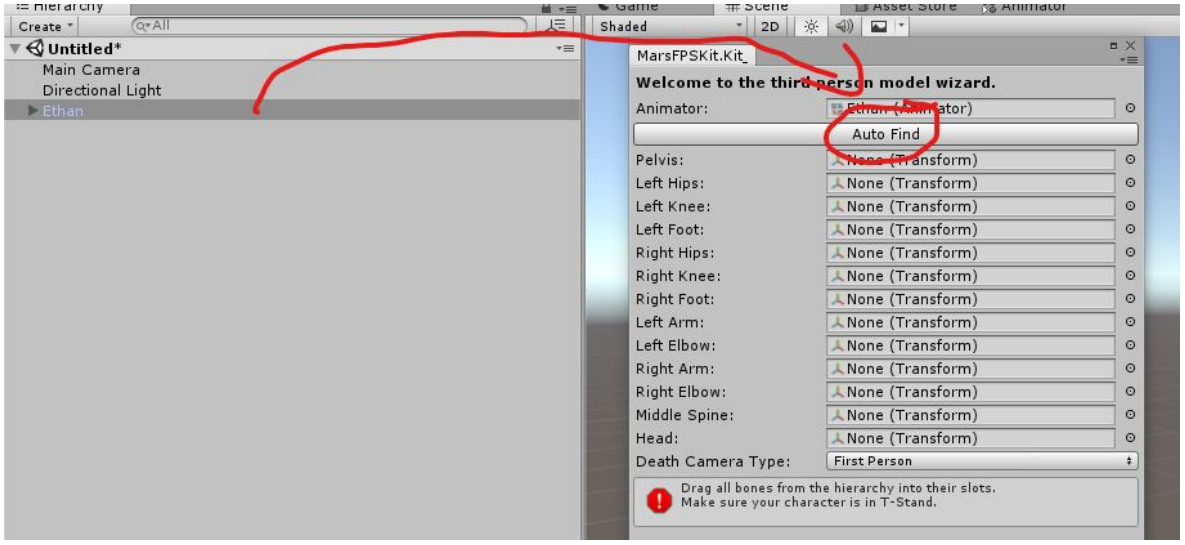
1. Set its animation type to "Humanoid", so Mecanim can retarget the animations
2. Configure the avatar to look like this:



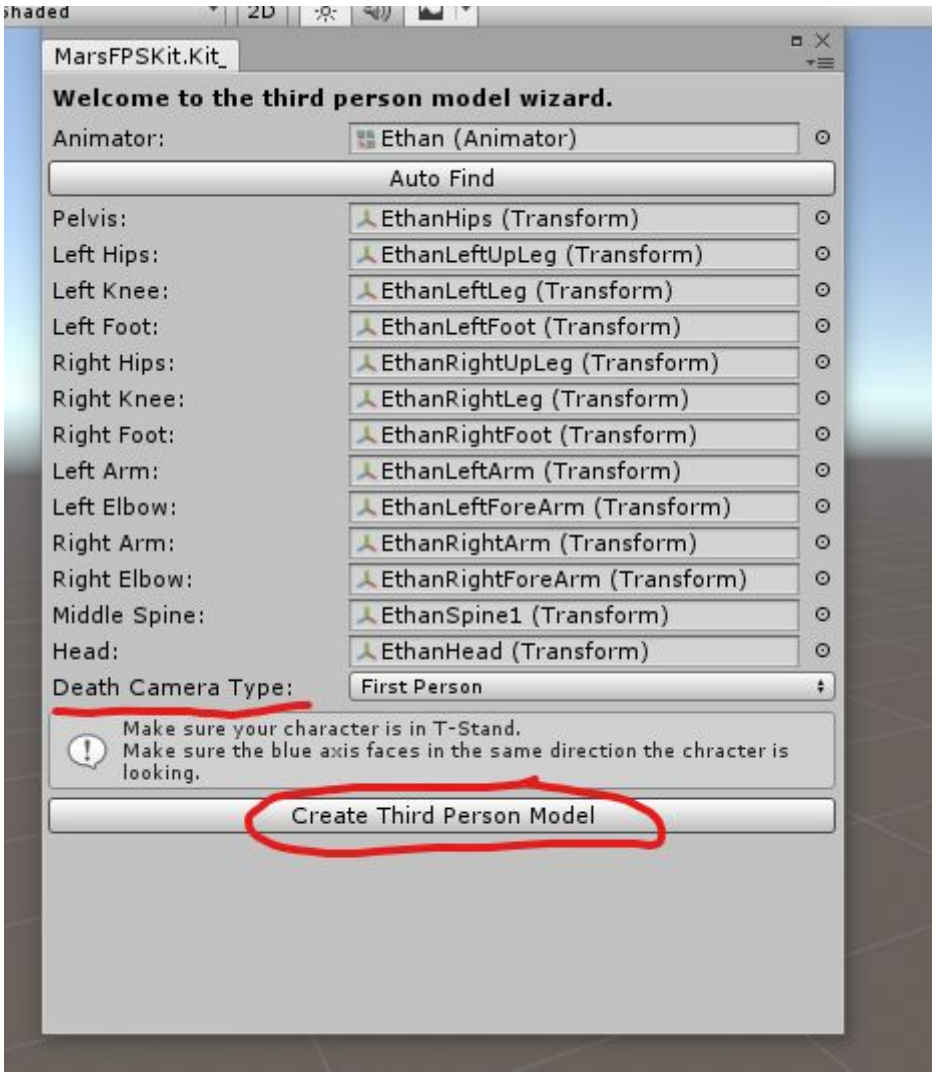
As of 0.5.6.0 the remaining part is extremely easy and fast!
First, drag your model into the scene and open the “Setup Third Person Player Model” tool which can be found under “MMFPSE/Player”:



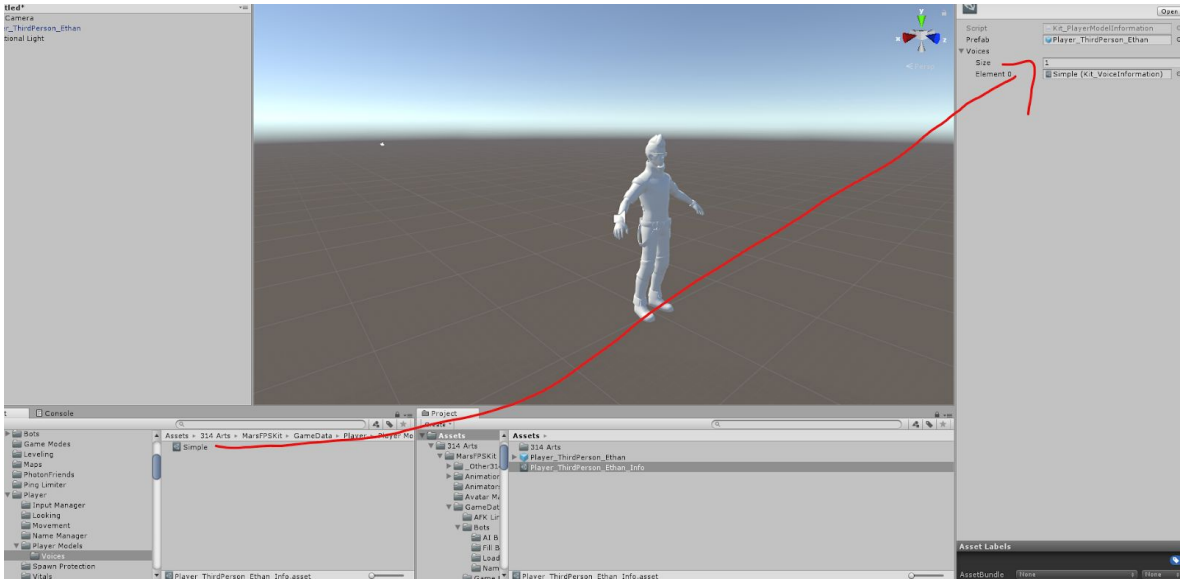
Then, drag your model into the “Animator” slot (it should be the root object) and press “Auto Find”. Every transform should be automatically assigned. In case they are not found (maybe because your bones are oddly named), you will need to assign them manually to the appropriate slot.



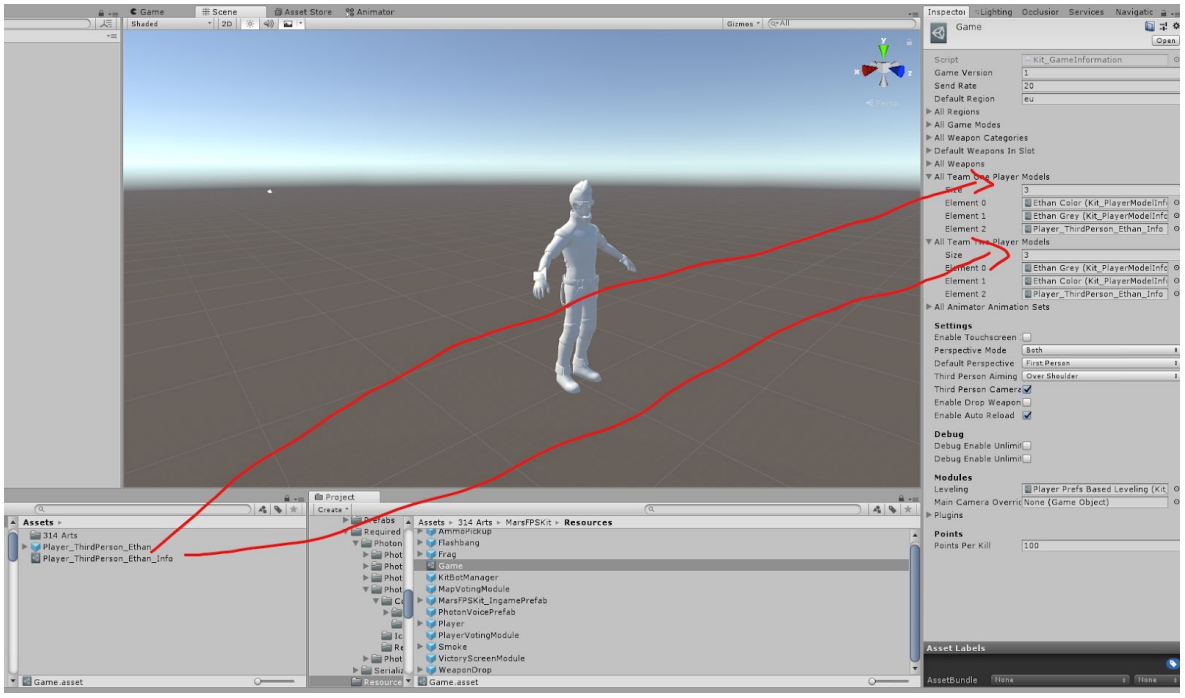
Then, select the Death Camera type you want to use and press the “Create Third Person Model” button:



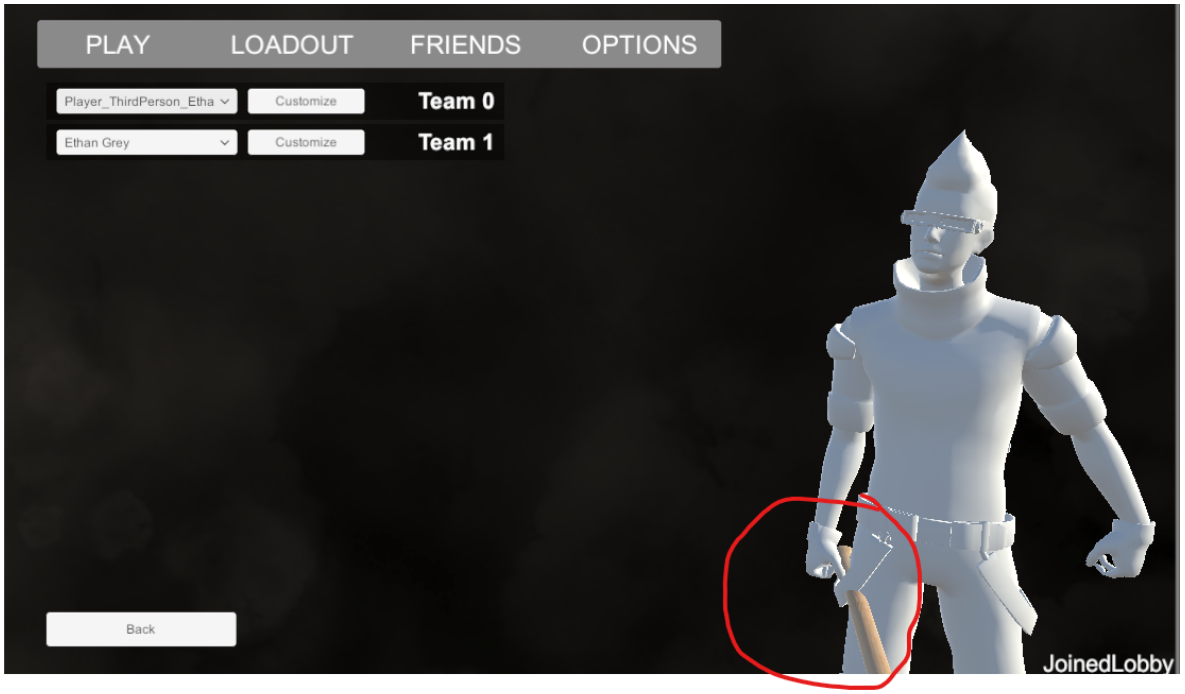
Then you will be asked where the model and the information file should be saved. Save them anywhere you want (inside your project). Afterwards we need to assign at least one voice to our information file that was just created.



Almost done! Now we need to assign our information file to the “Game” file, which can be found under “314 Arts/MarsFPSKit/Resources”:

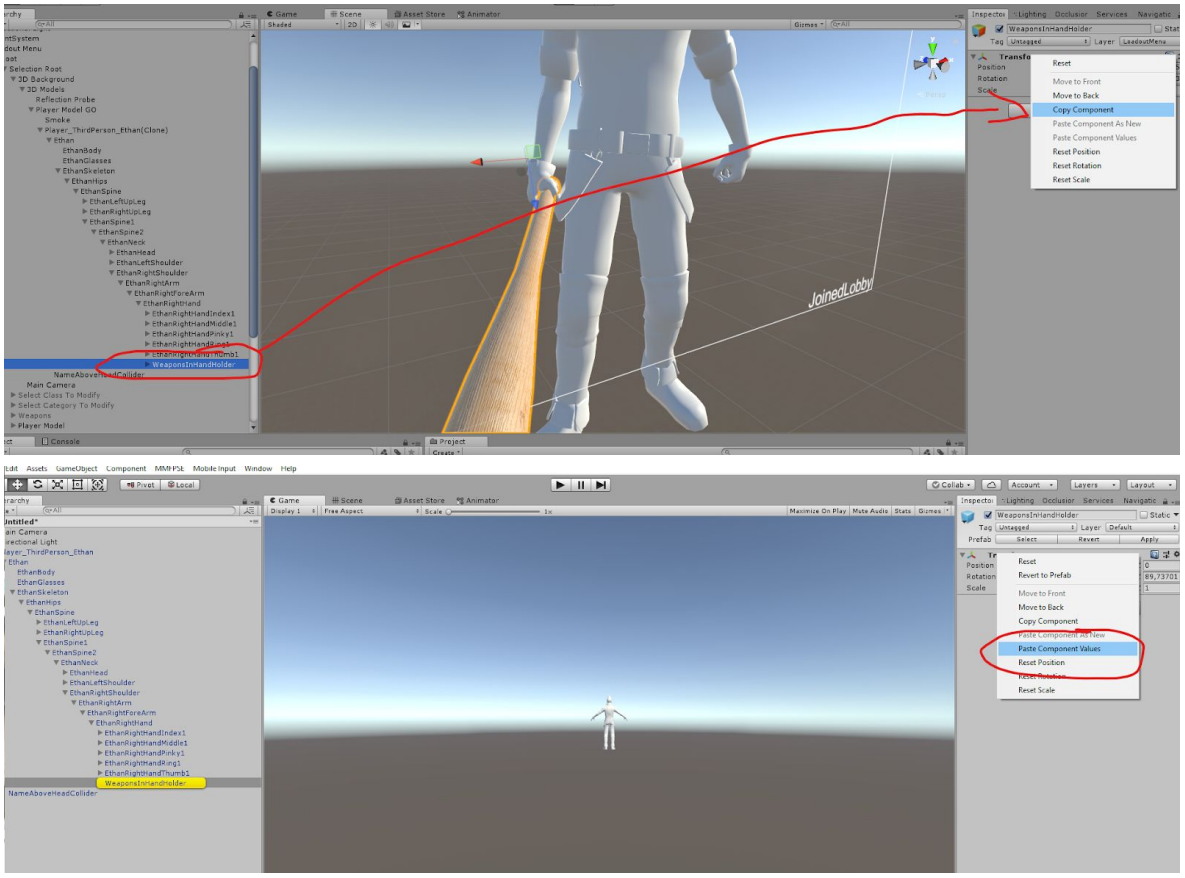


Done! At least technically. Now your model can be selected in the loadout menu, however, as you can see, it’s not configured 100% perfectly:



This can be easily fixed, by positioning and rotating the “WeaponsInHandHolder” like this and applying your changes to the prefab, by doing a right click on the “Transform”

component, pressing copy component and then pasting these values inside the prefab:



Done!
Of course, everything can be adjusted manually to your liking.

Modifying the UI

- Main Menu:
  The main menu is split in four parts:
  - Main menu
  - Login menu
  - Photon Friends menu
  - Options menu

  They all share the same canvas. Modifying them is very easy, just tinker around with the objects in the canvas and rearrange / modify as you like. Just make sure you reassign something new in the scripts if you deleted something. To avoid that, I recommend keeping things such as Texts in place and only changing their settings, but it is not required.
  Prefabs used in the main menu (Can be found in “MarsFPSKit/Prefabs/UI)
  - FriendEntryPrefab
    - Used for displaying a friend in the Photon Friends menu
  - Options Menu
    - Is already present in the main menu as a disconnected instance
  - RegionEntryPrefab

        - Used to connect to a different Photon Region in the Region menu
    - ServerBrowserEntryPrefab
        - Used to display a room in the “Browse Games” part of the menu
- In Game UI

  Everything is a part of MarsFPSKit_IngamePrefab, which can be found in “MarsFPSKit/Prefabs”. This prefab is also the base of the kit and required in every map’s scene.

  Things that are a part of it (colored parts are a module):
    - Pause Menu
    - Options Menu (Is a completely separate MonoBehaviour)
    - HUD
    - Kill Feed
    - Chat
    - Kill UI (points that pop up)
    - Victory Screen
    - End game Map Voting
    - Ping Limit
    - Afk Limit
    - Voting menu
    - Loadout Menu (Is also a completely separate MonoBehaviour)

  Every UI can be modified as usual. Just make sure to reassign UI components if you deleted something.

  Connected prefabs (Can be found in “MarsFPSKit/Prefabs/UI)
    - Everything in the folder “Game Mode HUDs”
    - ChatEntryPrefab
        - Used to display a chat message inside the chat UI
    - KillFeedEntryPrefab
        - Used to display a “Who killed whom” message
    - MapVotingEntryPrefab
        - Used for the voting at the end of the round
    - PlayerMarkerRootPrefab
        - This is the arrow / name that floats over the head of a player
    - ScoreboardPrefab
        - This displays a player in the scoreboard in a non team based game mode
    - ScoreboardTeamPrefab
        - Same as the former except for team based game modes
    - VoiceChatEntryPrefab
        - This displays an actively talking player in the lower left (by default)
    - VotingSelectionEntryPrefab
        - This displays the voting if a player starts a voting process
    - WeaponCustomizationDropdownSlotPrefab
        - Used to change attachments in the loadout menu when you click on customize

Update Photon

To update PUN2 and Photon Voice 2, do this:

1. Remove the “Photon” folder, which can be found in the “314 Arts\MarsFPSKit\Required” folder
2. Close Unity
3. Open Unity
4. Import “Photon Voice 2” WITHOUT all folders labeled as “Demo”
5. The update is done.
6. Optional: Move “Photon” folder back to the “Required” folder.

Photon is now updated. However, a few changes are made to Photon by the kit.
To keep these, do this:

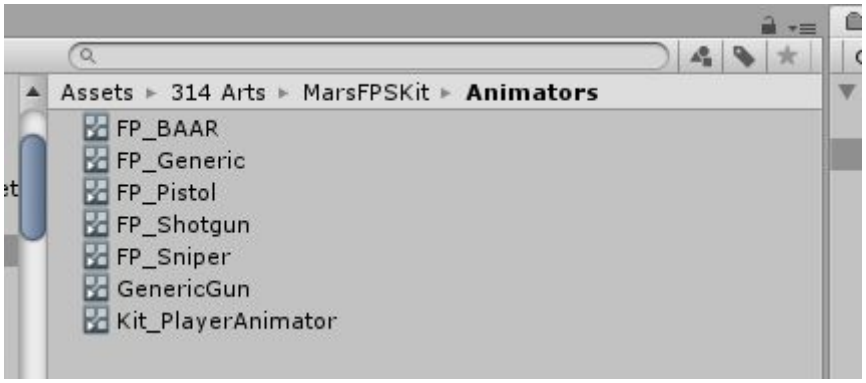
The change:

1. Open PhotonVoiceView.cs
2. Go to “SetupSpeaker”
3. Comment out “audioSource.spatialBlend = 1.0f; so that it looks like this:

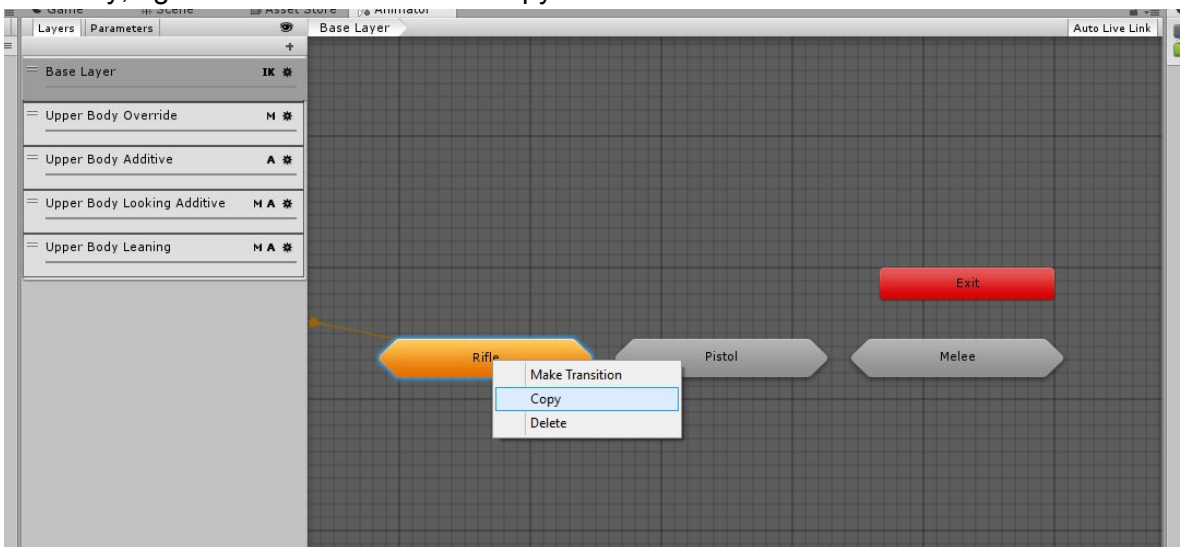
```
private bool SetupSpeaker(Speaker speaker)
{
    if (speaker == null)
    {
        if (this.Logger.IsWarningEnabled)
        {
            this.Logger.LogWarning("Cannot setup a null Speaker");
        }
        return false;
    }
    // get AudioSource and set spatialBlend
    AudioSource audioSource = speaker.GetComponentInChildren<AudioSource>();
    if (audioSource == null)
    {
        if (this.Logger.IsWarningEnabled)
        {
            this.Logger.LogWarning("Unexpected: no AudioSource found attached to the same GameObject as the Speaker component");
        }
        return false;
    }
    //audioSource.spatialBlend = 1.0f;
    return true;
}
```

Adding a new third person animation set

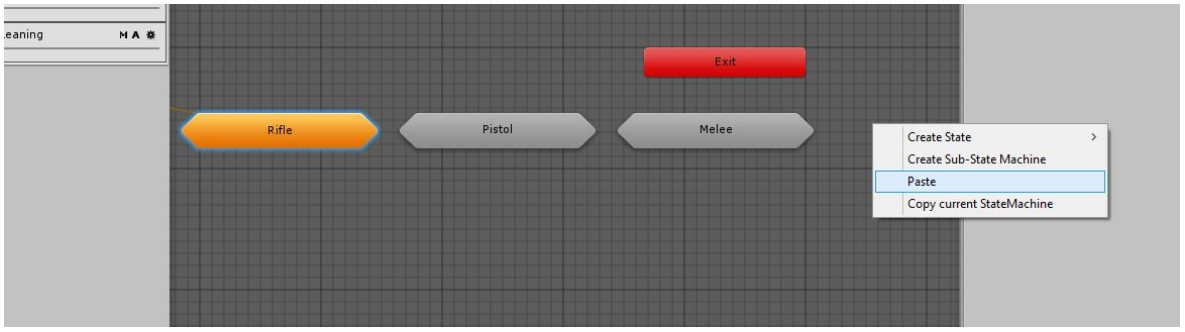
As of 0.5.2.0, adding a new third person animation set is EXTREMELY easy!
First, open the player animator “Kit_PlayerAnimator”, which can be found in “314 Arts/MarsFPSKit/Animators”.

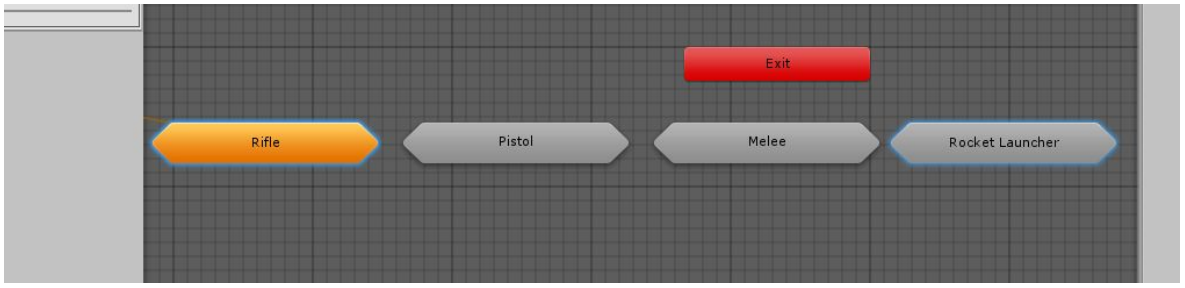


Secondly, right click on “Rifle” and hit copy.



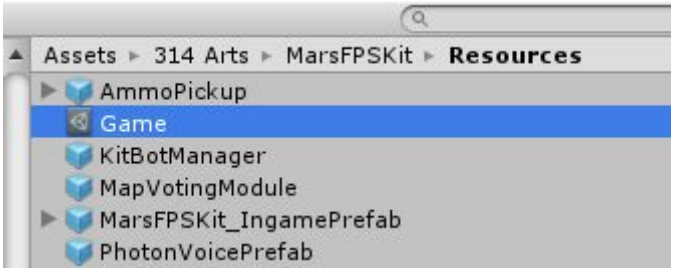
Thirdly, right click on an empty space and hit paste. Rename that pasted Statemachine to however you want to name it.



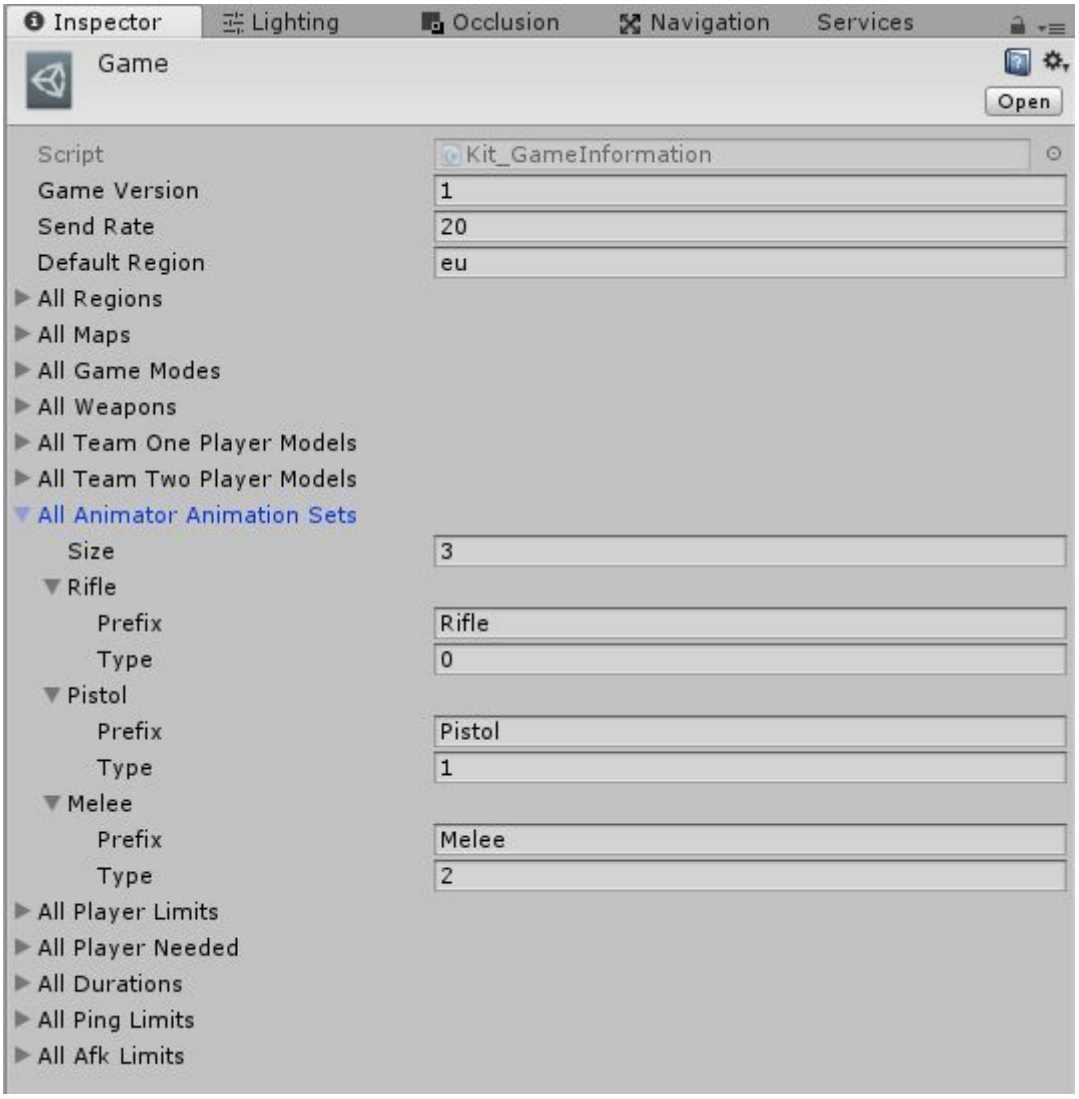


Then, select the “Game” asset, which can be found in “Assets\314

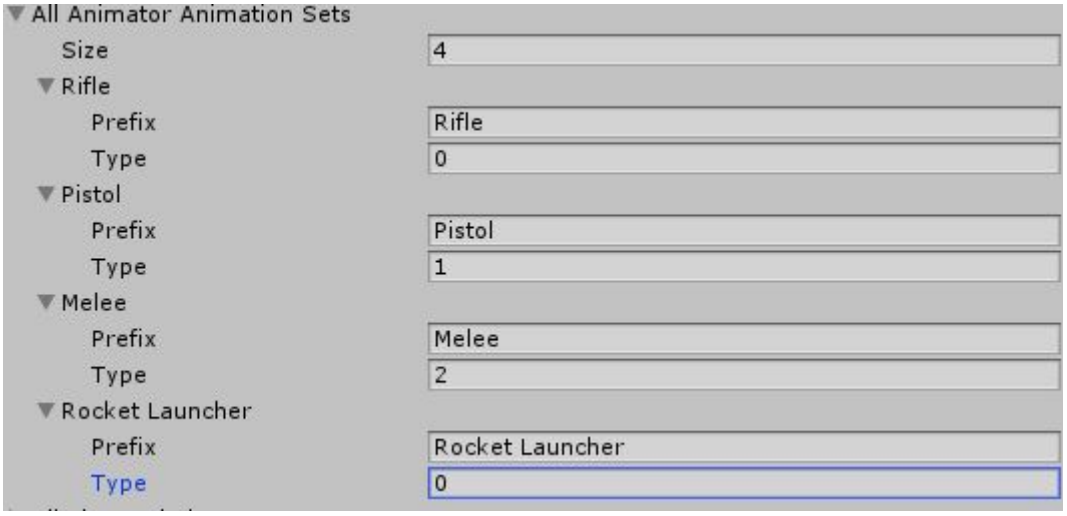
Arts\MarsFPSKit\Resources".



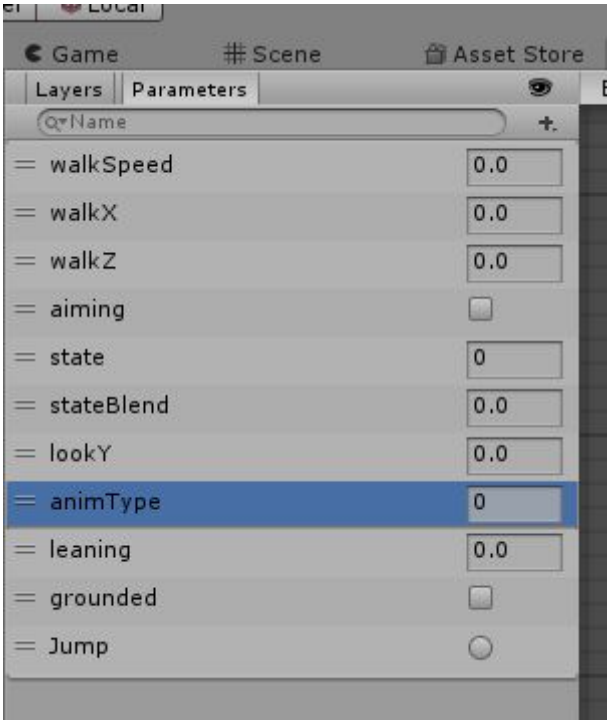
Under "All Animator Animation Sets", extend the length by one (example: 4 instead of 3) and in the new entry, set the Prefix to whatever you named the new Statemachine earlier.



->



The “Type” property is used for the looking layer “Upper Body Looking Additive” and will be set to the animator variable “animType” when the animation set is selected.

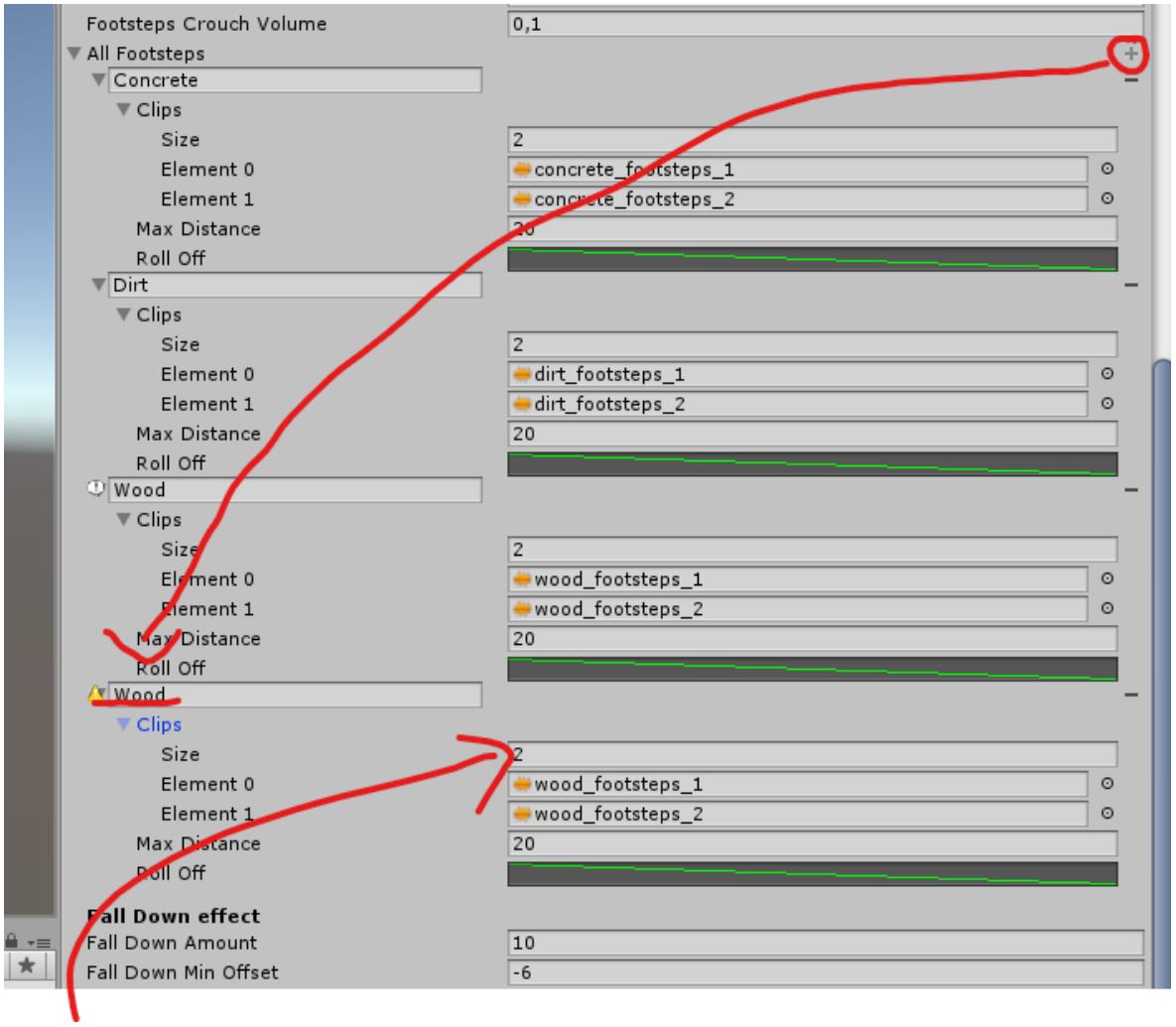


Adding new Footsteps

Footsteps are determined by tag of the object that you are walking on (except Terrains, for more on Terrains, see here).

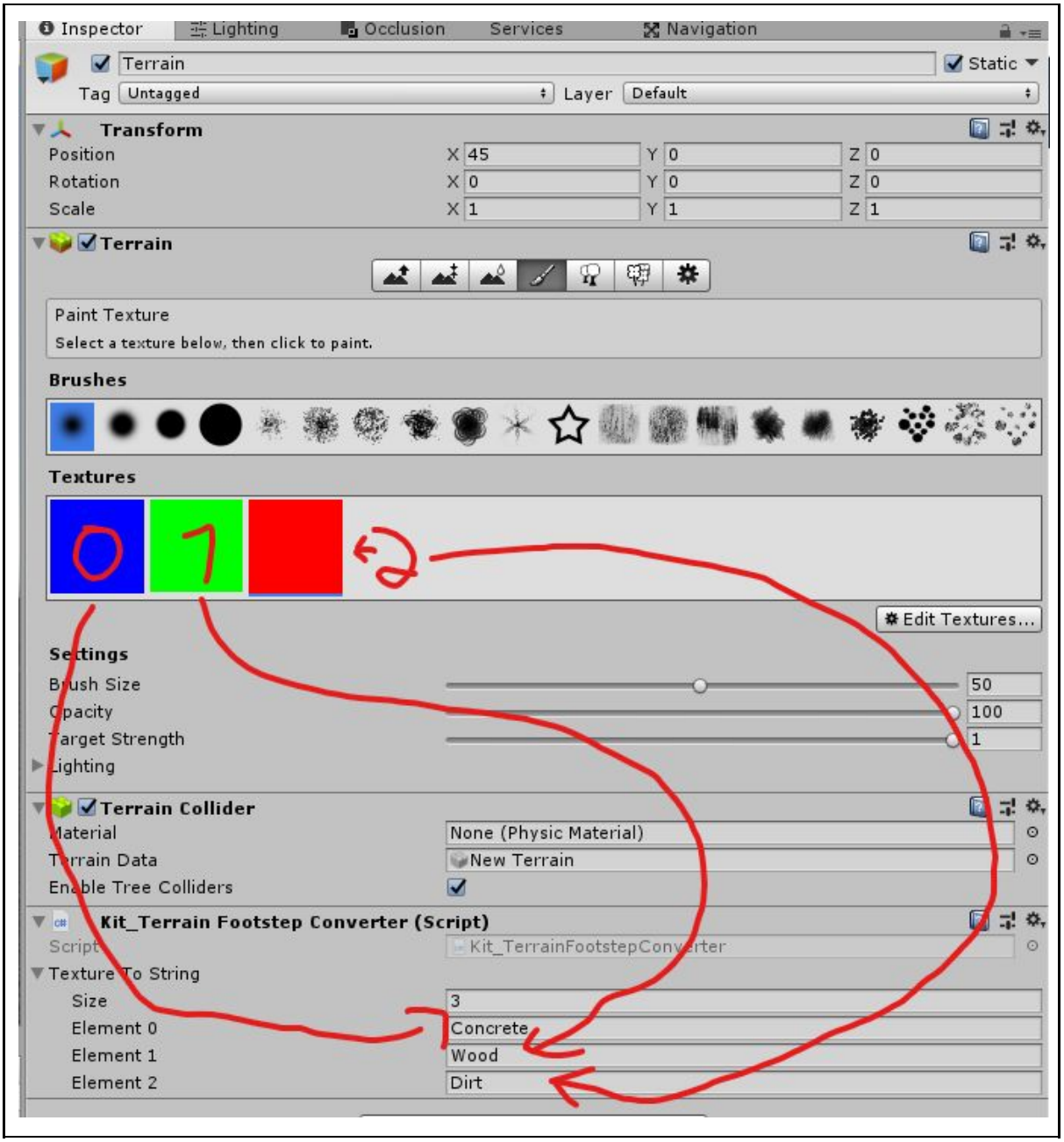
Adding a tag to the kit’s footstep system is really easy. It can be found on the “GenericBootsOnGround” object in “314 Arts\MarsFPSKit\GameData\Player\Movement”. To add a new supported tag, click the “+” button here and enter the tag into the marked

field, then you can drag in your new footstep sounds:



Terrain Footsteps

Footsteps on Terrain are determined via Texture to Tag script (called: “Kit_TerrainFootstepConverter”). Add it to your terrain and set the length of the array to the amount of textures you have on your terrain. The tag that the footstep system uses in then determined by the script, like this:



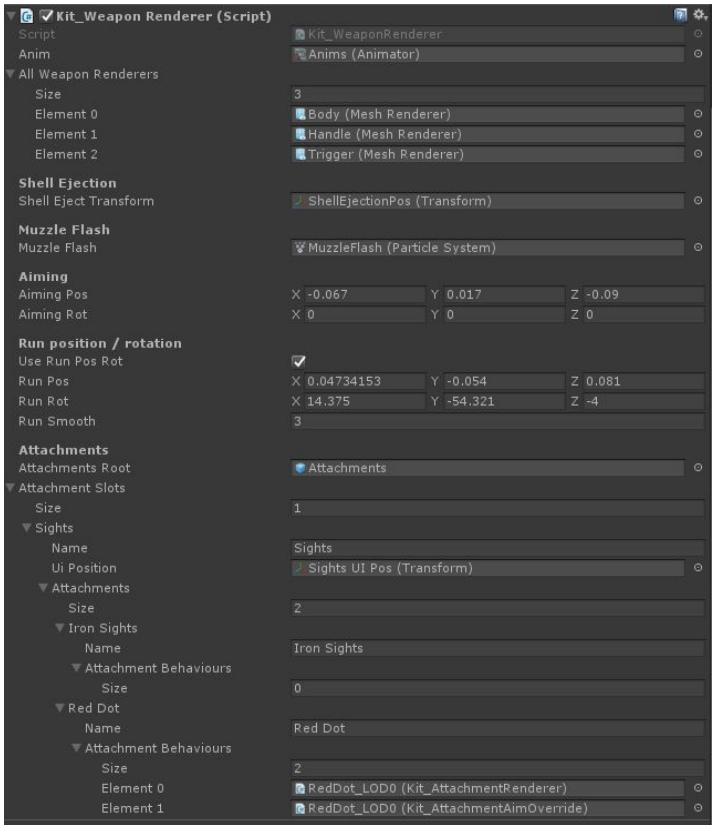
Extended Information

Weapon Setup

In the Mars FPS Kit, a weapon consists of:

- First Person Prefab
- Third Person Prefab
- Drop Prefab
- Kit_WeaponInformation
- Kit_WeaponBehaviour

The three prefabs are essentially the 'looks' of the weapon. These prefabs also carry the information for the attachments. It is VERY important that these line up. For example, it could look like this:
First Person Prefab:



- “Anim”

  This animator controls the animations of the weapon.

- “All Weapon Renderers”

  All renderers of this weapon should be assigned to this. Only default renderers, exclude attachments! These will be enabled when the weapon is selected and disabled when it is not.

- “Shell Ejection Transform”

  This is where the shell will be ejected, if it is enabled in the behaviour.

- “Muzzle Flash”

  This is a particle system that, if assigned and not disabled by attachments, will be played when the weapon is shot.

  NOTE:
  If you want to use multiple particle systems, use on system as the “master” and make all other particle systems children of this master in the hierarchy.

Then assign the master to the slot.

- “Aiming Pos/Rot”

  This is the default goto position and rotation in local space for ADS. It can be overridden using attachments. More about that here.

- “Use Run Pos Rot”

  If it is set to true, the weapon will be rotated to “Run Pos” and “Run Rot” with the speed multiplier of “Run Smooth”.

- “Attachments”:
  - “Attachments Root”

    This will be set to active / inactive depending on whether the weapon is currently in use (On screen). If you don't use attachments, you can leave this blank
  - “Attachment Slots”

    An attachment slot is a place where an attachment can be assigned. For example this could be “Sights”, “Under Barrel”, “Shaft”, “On Barrel”, etc..
  -
    - “Name”:

      This is the display name of this slot

    - “Ui Position”:

      This is where the Dropdown UI element for this slot will be displayed in the loadout menu. It is only required on the first person prefab but due to shared classes, will also be visible on the third person and the drop prefab. You can leave it blank there, it will do nothing.

  - “Attachments”

    A slot will be populated with the actual attachments. This could be various sights such as a Red Dot, a Micro T1, a sniper scope, a Kobra sight, etc or other stuff such as a suppressor or an extended barrel.
  -
    - “Attachment Behaviours”

      These define how an attachment acts. If it is empty, the attachment will do nothing, which can be useful if a slot is empty.

- Currently available behaviours:
  - “Aim Override”



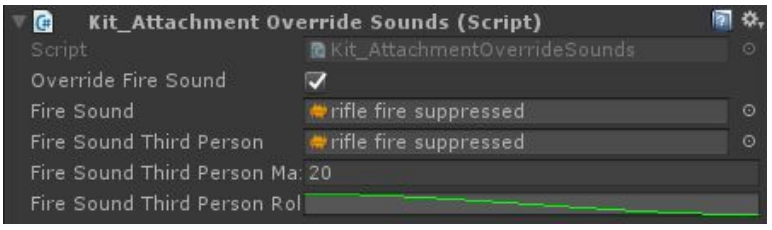
The “Aim Override” will override the “Aim Pos” and “Aim Rot” as described above. Useful for sights.

  - “Disable Muzzle Flash”



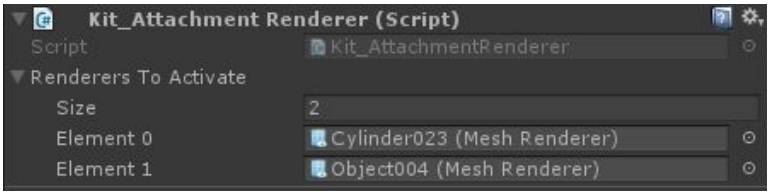
The “Disable Muzzle Flash” will disable the muzzle flash. Useful for suppressors.

  - “Override Sounds”



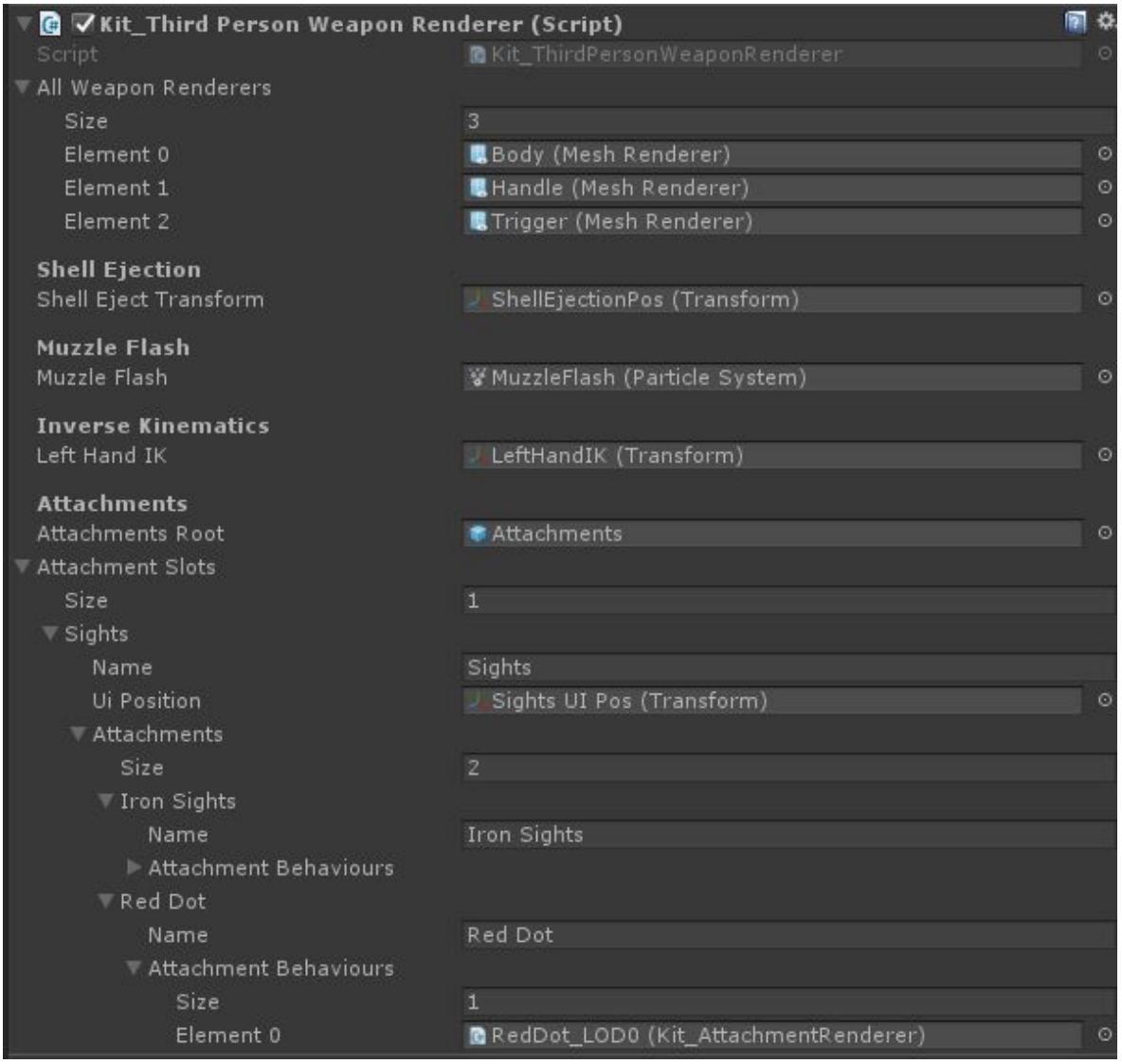
The “Override Sounds” will override the fire sounds and the third person falloff curve and range. Useful for suppressors.

  - “Renderer”



The “Renderer” will enable the given renderers if the attachment is selected. Useful for any type of attachment.

Third Person Prefab:



As you can see, it is mostly the same as the first person prefab. It mostly behaves the same except:

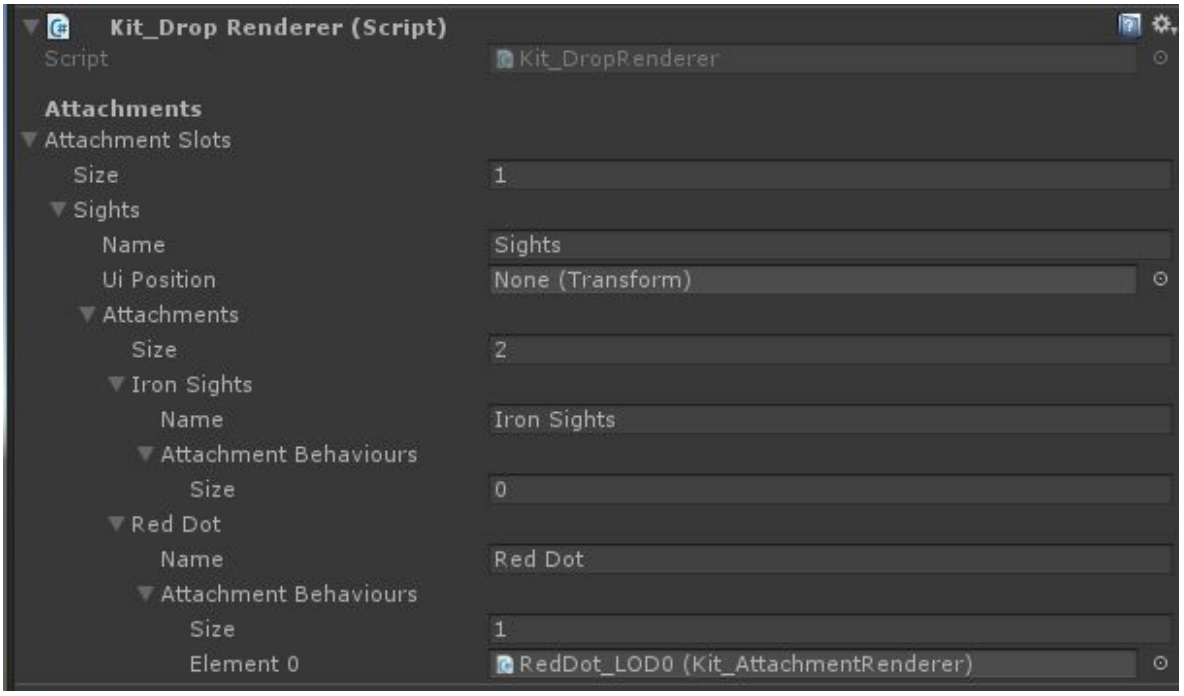
- “Inverse Kinematics”

  “Left Hand IK”: If this is assigned, the left hand will be positioned at the assigned transform using Mecanim. It is very useful to match the third person animations to the weapons.

- “Attachments”

  Certain attachment behaviours will not do anything in third person:
  - “Aim Override”: As there is no re positioning in third person, it will not do anything.

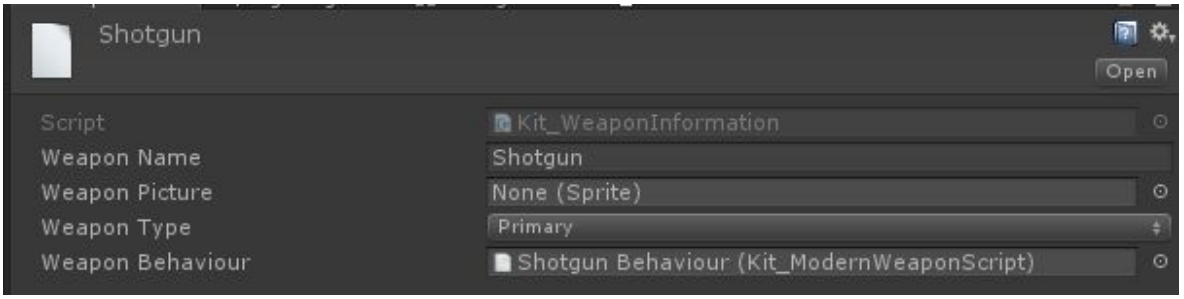
Drop Prefab:



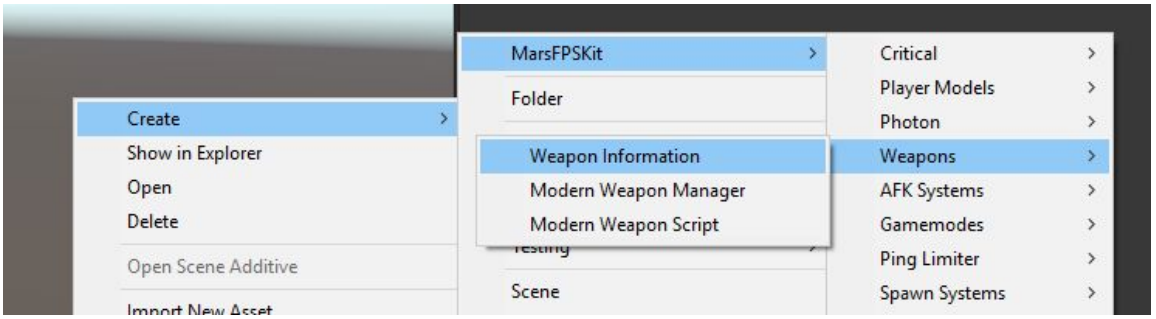
As you can see, it only has the attachments. Weapon drops don't do much except being there and looking correct.

NOTE REGARDING ATTACHMENTS:
Make 100% sure that First Person, Third Person and Drop prefab line up 100% in Slots and Attachments. Otherwise you will see errors and incorrect behaviour. The attachment behaviours can (and should be) different.

Kit_WeaponInformation:



This is a Scriptable object and can be created with a right click in the project and selecting:



- “Weapon Name”:

  How is this weapon called?

- “Weapon Picture”:

  Preview picture of this weapon for the loadout menu

- “Weapon Type”:

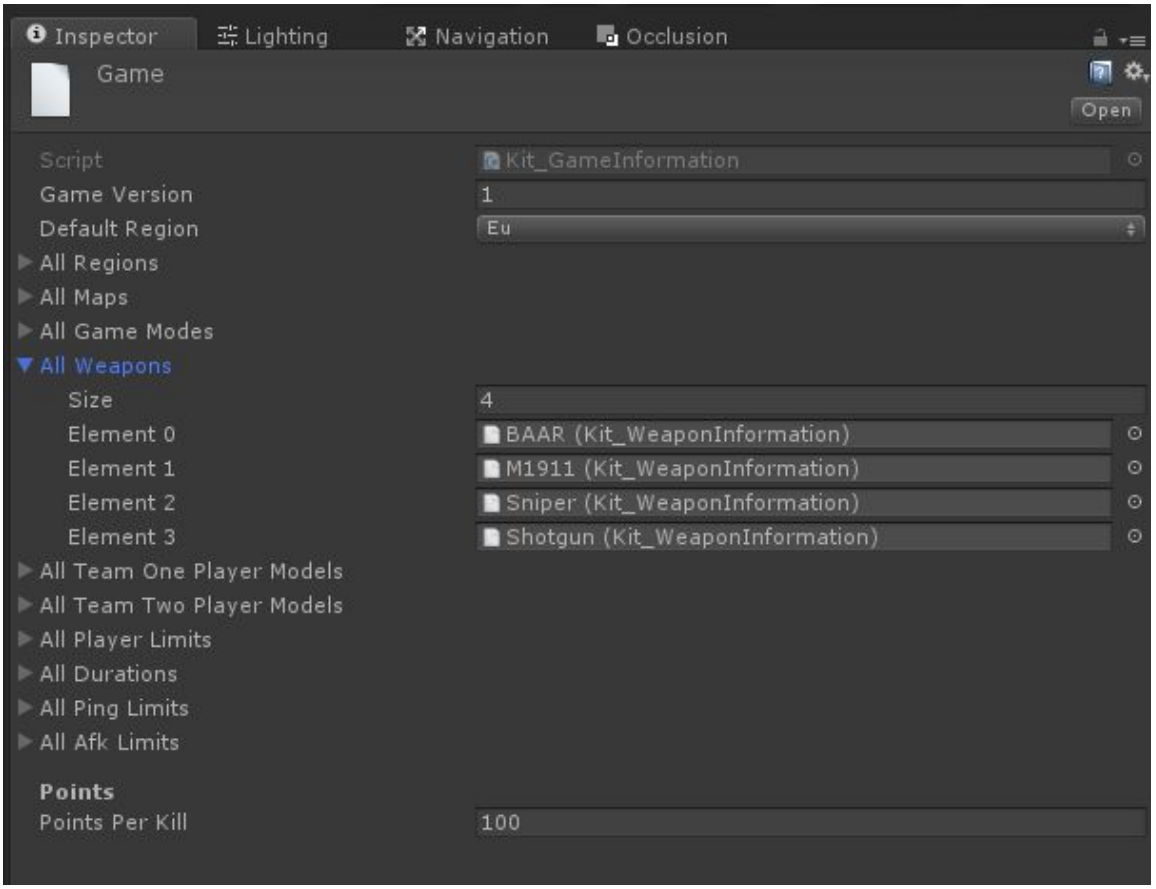
  Should this appear as a primary or secondary weapon?

- “Weapon Behaviour”:

  A scriptable object that defines how this weapon acts.

In order for a weapon to appear in game, it has to be in the “All Weapons” array of the “Kit_GameData” object, which can be found (by default) in:

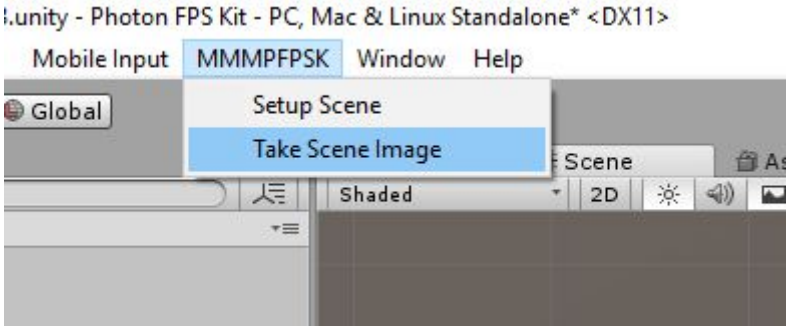
“Assets/MarsFPSKit/Resources/Game”. It should look like this:



Minimap

Setting up a scene for the minimap

You will notice that by default, the minimap in a scene will be blue. This can be easily changed by generating the needed minimap scene picture. It will only take a few seconds to do:
First, open up the wizard:



Adjust “Map Size” to fit your scene, like this:



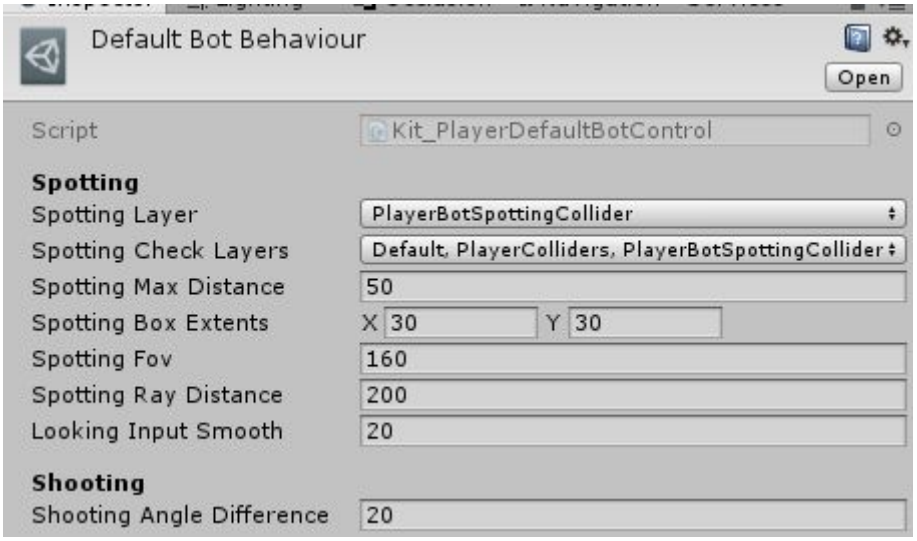
Select an appropriate texture size and press “Create!”. That’s it.

Bots

How they work

All input for the player is stored in a class and updated from the input component. This allows bots to “simulate” player input and use the same scripts as a normal player would. Always consider this when writing your own scripts!

Default AI Behaviour



Spotting:

Spotting Layer:
This is the layer that bots will “scan” for. The bot scan helper, which is a small box collider that is set to trigger, should be in this layer.

Spotting Check Layers:
These layers are checked with a raycast to check if a bot can see a player. This should include your game geometry aswell as the layer of the hit colliders of the player models.

Spotting Box Extents:
The box that the bot will use to check for players that he can see. Z is Spotting Max Distance.

Spotting Fov:
Essentially the camera FoV for bots. From the forward vector (of the camera), how big is the bot’s field of vision?

Spotting Ray Distance:
How far will the raycast mentioned in Check Layers go?

Looking Input Smooth:
Essentially how fast the bot’s simulated mouse input is. Increasing this will make bots a lot harder, decreasing this will make them easier.

Shooting:

Shooting Angle Difference:
At which angle will a bot start shooting (‘pressing left mouse button’)? To calculate this, the direct aiming vector (ideal shooting vector) is compared with the current forward vector of the bot’s ‘camera’.

Game Modes

Domination

Domination game mode setup is a bit more complex. Here are some infos to help you:
Flags:
To add a flag, add the “Kit_Domination_Flag” script to an empty game object.
Make sure that you drag all your flags under **ONE, SINGLE,** game object that holds them.
Refer to the example scenes to see how it’s done.
That is very important because the **HIERARCHY order** determines the ID of the flag!

Flag Nav Points (Normal Nav Point script):
Flag Nav Points are used by the bots in order to capture them. Place them inside the capturing zone (see the “DominationFlagPrefab” prefab for that). The IDs start at 1 (!!!) because ID 0 is used for backup nav points. ID 1 belongs to Flag 0, ID 2 belongs to Flag 1 (see above) etc…

Flag spawn points (Normal spawn point script):
Flag Spawn Points are used to spawn when a team owns a flag. Like the nav points they don’t start at 0 for the flags either. In contrast to the nav points, these start at 3, because 0-2 are used for other purposes.

Spawn IDs 0-2
Spawn ID 0 is used as backup spawns.
Spawn ID 1 is used for team 1 during the spawn countdown
Spawn ID 2 is used for team 2 during the spawn countdown

You can use the scene checker to see if you have not forgotten anything. You can find it under “MMMPFPSK/Check Scene” in the toolbar.

Platforms

WebGL

Notes

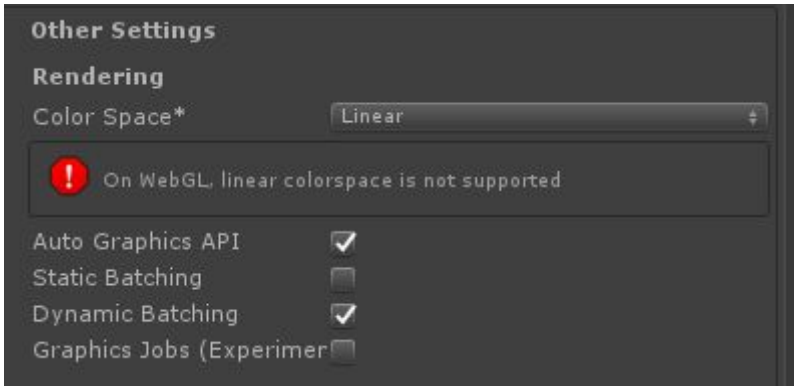
When you are targeting WebGL, you should note a few things:

- Change Color Space to “Gamma”
- Photon Voice is not supported on WebGL

Guide

Color Space

After you have changed your build target to WebGL, you need to change the color space from Linear:



To Gamma:



Photon Voice

To build for WebGL, you need to remove Photon Voice.
To do so, remove the “PhotonVoice” folder found in “314 Arts/MarsFPSKit/Required/Photon”.

Afterwards you will need to remove three scripts, which can be found in “314 Arts/MarsFPSKit/Scripts/GameLogic/VoiceChat”:
Kit_PhotonVoiceBehaviour
Kit_PhotonVoiceChat
Kit_PhotonVoiceChatUI

Now you should be able to build. If you cannot, contact us (methods below)

Compiling

Since WebGL is an AOT platform, you will need to use IL2CPP. For that you need a C++ compiler installed. I recommend installing this one.

Running your build

To run your build, simply start the index.html file. In Chrome, this will not work (locally). Read more about it here and how to get it running on Chrome (locally).

WebGL FAQ

My build crashes with an invalid function pointer error

Most likely your Unity version has a bug with “Strip Engine Code”. Disable that option in the Player Settings to avoid this error. Upgrading to a newer Unity version may also solve this issue.

What is the memory requirement?

At least 512 MB are required for the kit to function in stock state for WebGL.

Special Controls

Leaning

- Lean Left: Q
- Lean Right: E

Voice Chat

- Push To Talk: Left Alt
- Global Voice: L
- Team Voice: K

Integrations

Photon Voice

The Photon Voice integration is already inside the Kit and requires no further work.
A note regarding the AppID: Using the same AppID you are using for the main game will result in doubled CCU. This is due to Photon Voice acting on its own, independently of the main game. E.g. when you join a PUN room, you will also join a Photon Voice room.

First Person View 2

Drunken Lizard Games' First Person View 2 is the proper solution for avoiding first person weapons clipping through the environment. Classic solutions like a dual camera setup have a lot of downsides, including, but not limited to:

- No shadows on first person weapons
- Performance downgrade
- Breaks some image effects (like TXAA)

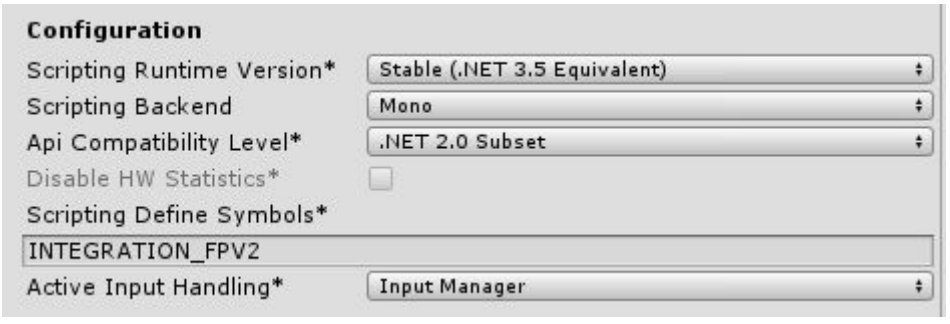
This is, why the kit features a built in integration for FPV2. It requires some changes to be made.

Getting started

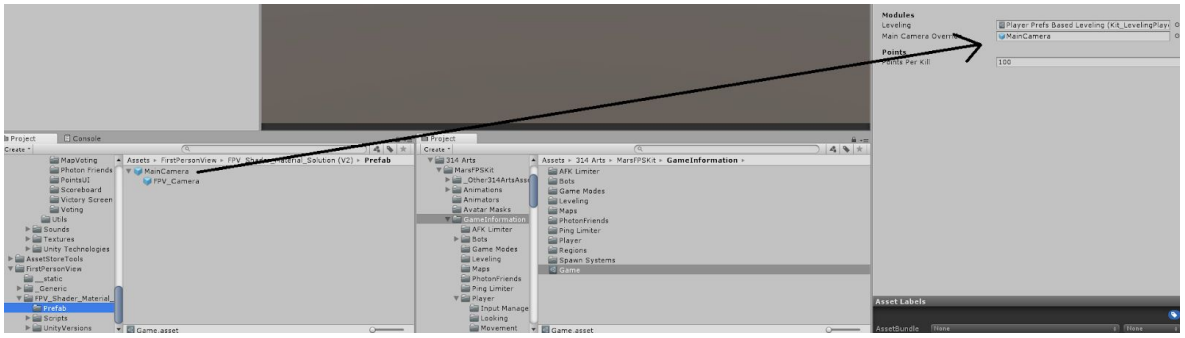
First, import FPV2 from the Asset Store.
Then, follow the setup instructions found in the FPV2 manual.

Enabling the integration

Second, enable the integration by defining the symbol "INTEGRATION_FPV2" in the Player Settings:



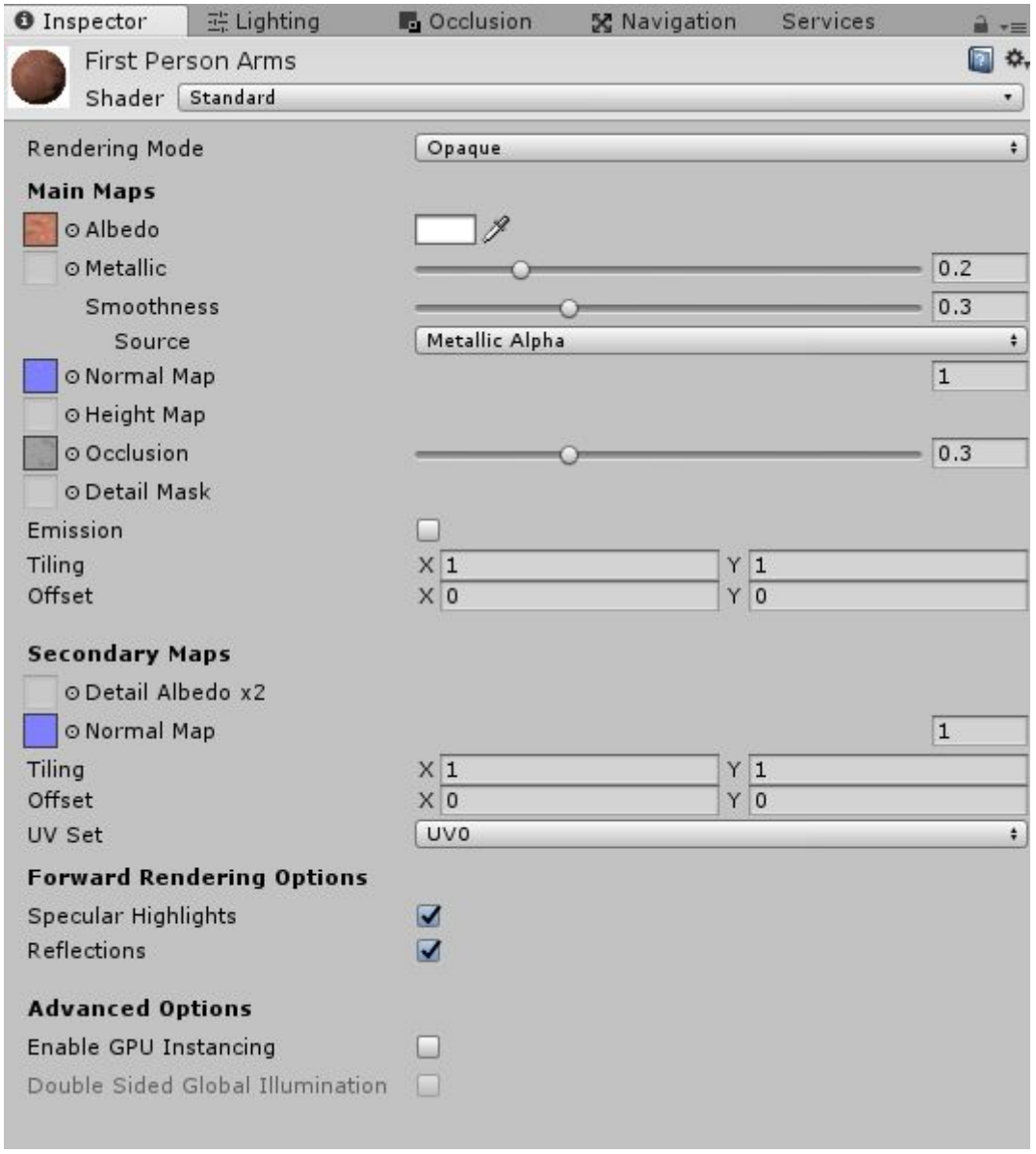
Then, assign the “MainCamera” prefab from the “Assets/FirstPersonView/FPV_Shader_Material_Solution (V2)/Prefab” folder to the “Main Camera Override” in the “Game” file:



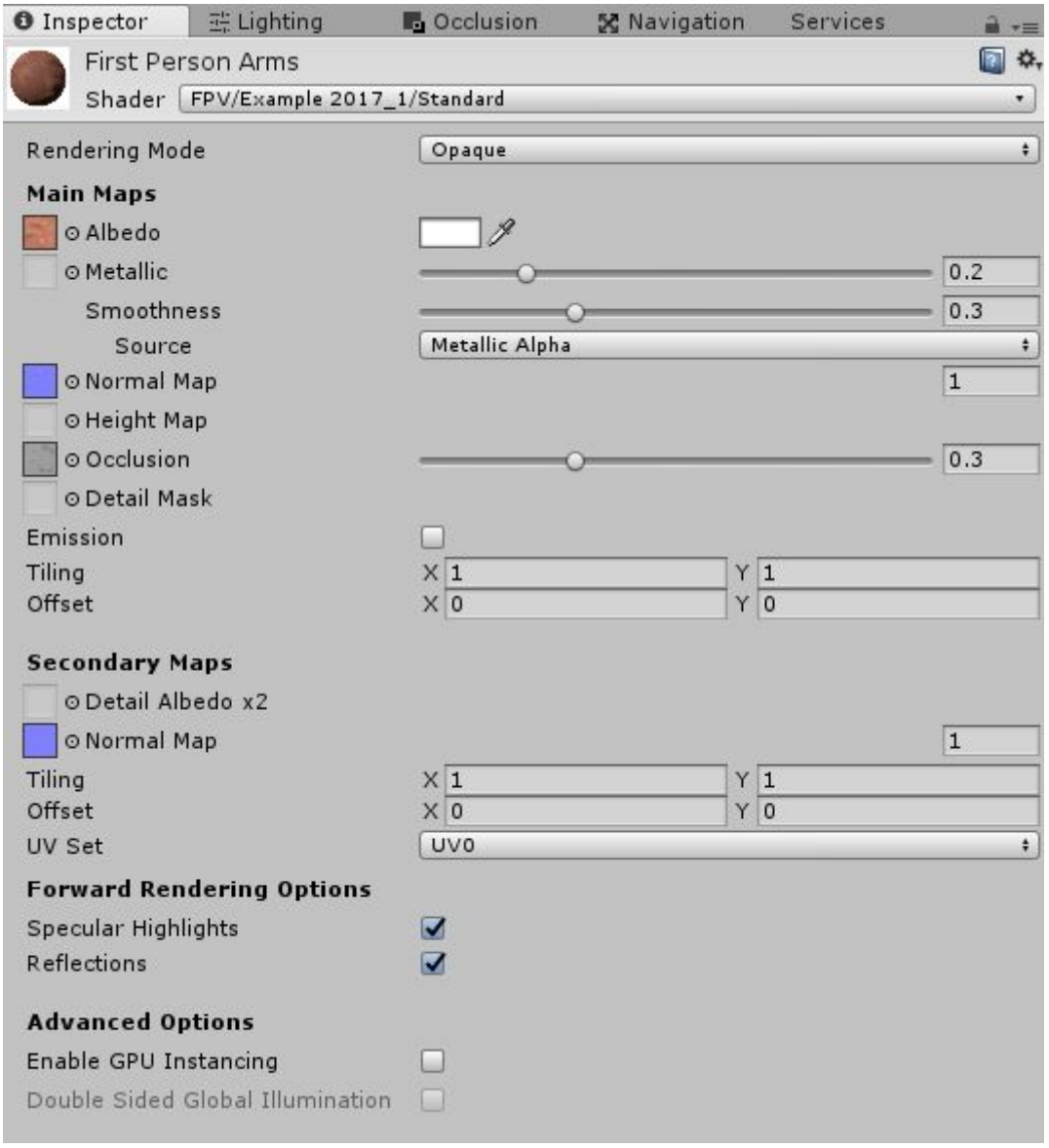
Finally, uncheck the “Minimap” and “LoadoutMenu” layer on the prefab’s camera in the “Culling Mask”.

Changing shaders

Thirdly, you need to change the shaders of the first person weapons and arms, to FPV variants. If you need to use custom shaders, refer to the FPV2 manual.
Arms example, from:


Inspector
Lighting
Occlusion
Navigation
Services
First Person Arms
Shader
Standard
Rendering Mode
Opaque
Main Maps
Albedo
Metallic
0.2
Smoothness
0.3
Source
Metallic Alpha
Normal Map
1
Height Map
Occlusion
0.3
Detail Mask
Emission
Tiling
X 1
Y 1
Offset
X 0
Y 0
Secondary Maps
Detail Albedo x2
Normal Map
1
Tiling
X 1
Y 1
Offset
X 0
Y 0
UV Set
UV0
Forward Rendering Options
Specular Highlights
Reflections
Advanced Options
Enable GPU Instancing
Double Sided Global Illumination

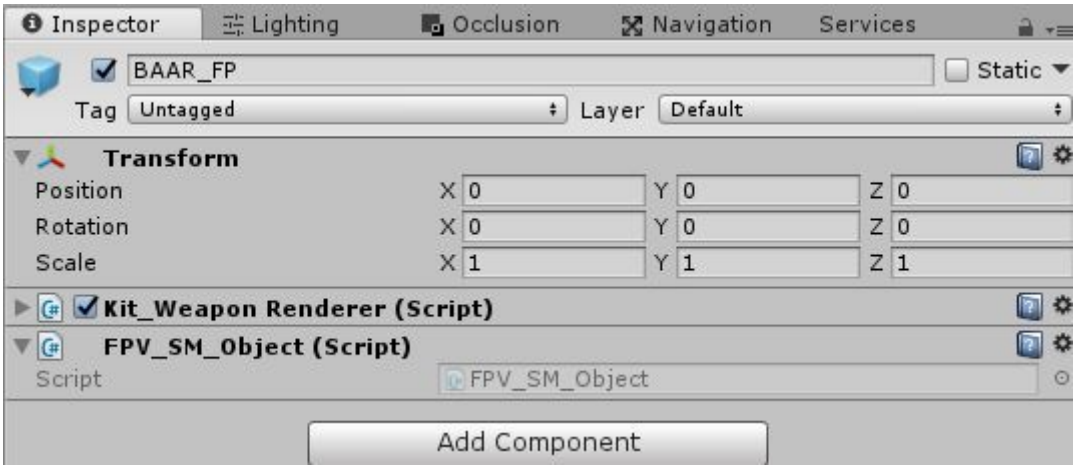
To:



Hint: Do not forget to use the correct Unity version shaders.
Second hint: You need to change the materials of all first person objects (By default: BAAR, Sniper, Pistol, Shotgun.
Third hint: Do not forget to change the materials of your attachments too.

Modifying the prefabs

At the very last, you need to add the “FPV_SM_Object” (It will show up as “FPV Object” when clicking Add Component) to all first person prefabs, meaning prefabs that (by default) end with “_FP”, like this:



Done

Now, play the project and check if all weapons work correctly. If that is the case, you're done!

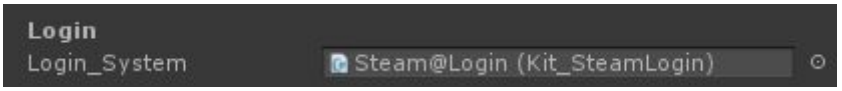
Steam

MMMPFPSK features an in house Steam integration. It will fetch information from Steam such as the username and friends. It will enable users to invite their friends to their respective Photon room.

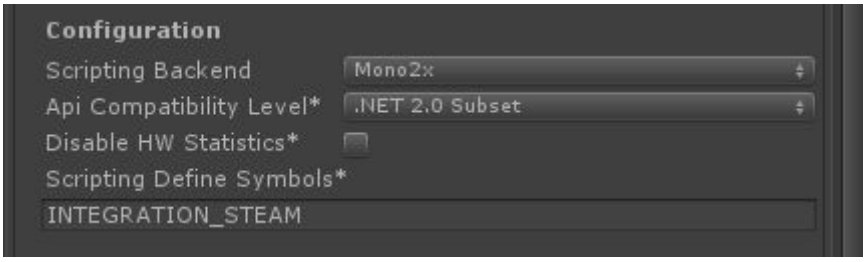
Getting Started

Main

1. Extract the "Steam Integration.zip" file in the "MarsFPSKit/Integrations" folder
2. Copy the "steam_appid.txt" file (search it in the search bar) to your Project root (where "Assets", "Library" and "ProjectSettings" is).
3. Restart Unity
4. Drag the "Steam@Manager" in the "MarsFPSKit/Integrations/Steam Integration/Prefabs" (Referred to as "Prefabs folder" in the following steps) folder in the Main Menu scene
5. Do the same with the "Steam@Login" prefab and assign the Kit_SteamLogin script to the "Login_System" field in the Main Menu script



6. Disable the "Login" Game Object in the Main Menu under "Canvas/Main Menu" as it is no longer needed for Steam login
7. Define the scripting symbol "INTEGRATION_STEAM" in the Player Settings:



Friend Menu

The Steam Integration features an in game friends menu that allows you to join all your Steam friends that are currently playing your game. To set it up follow these steps:

1. Assign “Steam@PhotonFriends” to the “PhotonFriends” field in the MainMenu script
2. Disable the “Add Friend” object in “Canvas/Main Menu/Friends/Background Window/Add Friend
3. You are ready to go!

Using your own AppID

If you have acquired an AppID using Steam Direct or using a different way (e.g. Old Greenlight project), all you need to do is to change it in the “steam_appid.txt” file in the project root and in the Steam@Manager prefab in “MarsFPSKit/Integrations/Steam Integration/Prefabs”.

Features

- Use Steam account information (Username)
- Join other players with Steam’s “Join game” function (Does not function with the default “480”/”Spacewar” AppID. Needs a unique AppID to function
- Invite other players to your room
- See all your Steam friends in the kit’s friends menu

Recommended Assets

Animations

It is recommended to replace the third person player animations. These assets will do a great job:
[Rifle Animset Pro]

Rifle Crouch And Prone
Pistol Animset Pro

FAQ Help

My weapons are clipping through level geometry?!

There are multiple solutions to this problem.

1. Use a dual camera setup. One camera renders the level geometry and the other one renders the first person weapons. I really do not recommend this for a couple of reasons.
    a. Many image effects will break with this setup
    b. It will decrease performance
    c. Level geometry will not cast shadows on weapons
2. Use First Person View 2. It is a really good asset for only $15 and will find application in all your next first person projects. Get it here. Integration instructions here.
3. Resize your weapons to make them fit inside the Character Collider. It will mean that you will have to adjust walking and running animations though.

The Loadout Dropdown Buttons are rotated.

This is most likely caused by rotating the “MarsFPSKit_IngamePrefab” object in the scene. Its rotation and position shall remain at 0,0,0. To move/rotate the scene camera use the “Spawn Camera Position” object.

How do I scale, rotate or move a weapon prefab (first, third-person or drop prefab)?

You should always create an empty game object and use that as the prefab itself. This should hold the scripts and have its position and rotation at 0,0,0 and scale at 1,1,1. The model itself (the one which has the animator on it in case of the first person prefab) should be directly under it in the hierarchy.

I replaced the hit indicator. Now it is not centered anymore.

If you did not touch the UI itself (positions and pivot), the reason for this is most likely your replacement image. You need to make sure that it's completely ready to be rotated around its center. Check out the default image to see how it should look.

[...] is inaccessible due to its protection level.

This error is most likely caused by using an outdated Unity version. For proper function, please only use Unity 2018.1 and up.

My snipers are inaccurate.

To make your snipers fully accurate, go to its behaviour, open the tab "Bullet Spread" and make sure Aim base is close to 0 (0 for 100% accuracy) and Aim velocity base add is 0.

I spawn as a construction worker.

You imported the Photon Demos. As stated in the Asset's description, the kit comes ready with Photon. If you need to update, do not import Photon's demos.

My customization UI Dropdowns are super small / super big

This is caused by having the scale of them offset.
Make sure that the UI object itself has a scale of 1,1,1 in (WORLD SPACE!)!
You can check, by dragging it outside of the prefab. Unparent the actual attachments and set its scale to 1,1,1; then reparent them and put it back where it belongs.

Need help?

Do you have a problem that you cannot solve using this document? You can seek help on Armedunity, we have a specific team for this kit and a big community that can help you with

non kit related problems (in Unity!). If you need immediate help or you just want to join our community, join our Discord here!